

Министерство образования и молодежной политики Свердловской области
Государственное автономное профессиональное образовательное
учреждение Свердловской области «Уральский политехнический колледж-
Межрегиональный центр компетенций»
(ГАПОУ СО «Уральский политехнический колледж - МЦК»)

Допустил к защите
Заведующий отделением
_____ В.О. Дарьева
«_____» _____ 2026 г.

**РАЗРАБОТКА ПРОЕКТИРОВАНИЯ И
ПРОГРАММИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ
МОНИТОРИНГА БАНКОВСКИХ ОПЕРАЦИЙ ДЛЯ ПАО
«СБЕРБАНК»**

Пояснительная записка к дипломному проекту
ДП.09.02.05.07.19.24.ПЗ

Специальность 09.02.07 Информационные системы и программирование
(по отраслям)

Руководитель проекта
_____ И.И. Коломейцев
Консультант экономической части
_____ Д.Д. Пургина
Нормоконтролер технической
документации
_____ О.Н. Тобина
Студент группы 22/ИС-491кв
_____ С.И. Пономарев

г. Екатеринбург, 2026

Содержание

ВВЕДЕНИЕ	3
1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ.	5
1.1. Описание предметной области.	5
1.2. Описание предмета автоматизации.	20
1.3. Анализ существующих разработок.	25
1.4. Используемое программное обеспечение.	30
1.5. Описание используемых языков программирования.	32
2. ПРАКТИЧЕСКАЯ ЧАСТЬ.	33
2.1. Постановка задачи.	33
2.2. Выбор сред разработки.	33
2.3. Вайрфрейм программного продукта.	34
2.4. Структура программного продукта.	35
2.5. Этапы разработки программного продукта.	36
2.6. Модели бизнес-процессов.	37
2.7. Функциональная диаграмма IDEF0	39
2.8. Диаграмма потоков данных DFD	41
2.9. Диаграмма «сущность-связь».	42
2.10. Руководство пользователя.	43
3. ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ ЭФФЕКТИВНОСТИ ПРОГРАММЫ.	52
4. ОХРАНА ТРУДА.	58
ЗАКЛЮЧЕНИЕ.	64
Приложение А. Листинг приложения	65
Приложение Б. Техническое задание	94

ВВЕДЕНИЕ

Современный сектор банковских услуг характеризуется динамичными преобразованиями, среди которых одним из наиболее значимых векторов развития выступает необходимость всесторонней оптимизации процессов управления и мониторинга финансовых операций. В ходе проведения исследовательской работы и углубленного анализа практического опыта, была отчетливо осознана критическая важность для финансовых учреждений не только обеспечения надежной защиты активов и конфиденциальности данных своих клиентов, но и поддержания абсолютной прозрачности всех осуществляемых транзакций. Это требование продиктовано как возрастающими ожиданиями потребителей в условиях цифровизации, так и усиливающимся регуляторным давлением, направленным на минимизацию рисков, таких как мошенничество и отмывание средств, и повышение доверия к банковской системе в целом. Таким образом, создание специализированной информационной системы, предназначенной для эффективного мониторинга банковских операций, перестало быть исключительно академической задачей, ориентированной на применение теоретических знаний. Оно трансформировалось в комплексный научно-практический вызов, предоставляющий уникальную возможность для глубокого погружения в проблематику, имеющую фундаментальное значение для устойчивого функционирования современной экономической системы. Подобные системы становятся краеугольным камнем в архитектуре безопасности и операционной эффективности, позволяя не только фиксировать, но и анализировать потоки данных для выявления аномалий и потенциальных угроз, тем самым способствуя поддержанию финансовой стабильности.

На протяжении всего периода практической деятельности было уделено особое внимание тщательному исследованию существующих технологических решений в рассматриваемой области, а также детальному анализу требований, предъявляемых к таким системам как конечными пользователями из числа банковского персонала, так и регулируемыми органами. Проектная работа позволила не только успешно интегрировать и применить обширный объем теоретических знаний в области информационных технологий и финансового анализа, но и существенно развить практические навыки программирования, архитектурного проектирования сложных систем, а также компетенции в области творческого подхода к решению нестандартных задач. Одной из ключевых задач являлось создание интуитивно понятного и эргономичного пользовательского интерфейса, обеспечивающего легкий доступ к функционалу как для внутренних сотрудников банка, ответственных за операционный контроль, так

Подпись и дата						
Имя	Подпись					
Взам или №						
Подпись и дата						
Имя №						
Изм.	Лист	№ докум.	Подпись	Дата	ПП.09.02.07.491.19.26 ПЗ	Лист
						3

В настоящем отчете будет представлен систематизированный анализ опыта, накопленного в процессе разработки упомянутой информационной системы для мониторинга банковских операций. Изложение материала охватывает каждый этап данного проекта, начиная с исходного этапа анализа функциональных и нефункциональных требований, включая выбор архитектурных решений и методов реализации, и заканчивая комплексным тестированием системы, призванным подтвердить ее надежность, производительность и соответствие заданным спецификациям.

Үнх № тогтол Подпись и дата Рзам или № Илч № дуга Подпись и дата						ПП.09.02.07.491.19.26 ПЗ	Лист
							4
	Изм.	Лист	№ докум.	Подпись	Дата		

1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ.

1.1. Описание предметной области.

В современном финансовом мире банковские операции — это неотъемлемая часть функционирования финансовых институтов, экономики в целом, а также механизма борьбы с мошенничеством, отмыванием денег и финансированием терроризма. Эффективное управление и контроль этих процессов обеспечивается системами мониторинга, которые используют современные технологии анализа данных, алгоритмы машинного обучения и правила бизнес-логики. Эта система предназначена не только для поддержки внутренних процессов банка, но и для соответствия юридическим и нормативным требованиям, установленным государственными и международными регуляторами.

Цель мониторинга — выявление подозрительных и потенциально мошеннических операций, предотвращение финансовых преступлений, обеспечение стабильности и доверия к банковской системе. Для достижения этой цели разработаны различные методы, модели и автоматизированные системы, которые позволяют обрабатывать и анализировать огромное количество транзакционных данных в реальном времени.

Основные понятия и определения

1. Банковская операция

Определение: Это любое действие, совершаемое клиентом или банком, связанное с управлением средствами или активами в рамках банковских услуг. Банковские операции включают в себя переводы, платежи, кредиты, обмен валют, операции с ценными бумагами и др.

Примеры:

Перевод средств со своего счета на счет другого клиента.

Оплата товаров и услуг через интернет-банкинг.

Взятие кредитных средств.

Получение наличных через банкомат.

Внесение денег наличными или переводы с карты на карту.

Ключевые особенности: Каждая операция содержит такие параметры, как сумма, валюта, дата, участники сделки, местоположение, устройство и канал выполнения.

2. Мониторинг операций

Определение: Процесс автоматической или ручной оценки проведенных транзакций с целью выявления подозрительных или незаконных действий. Основная задача — своевременно обнаружить признаки мошенничества, отмывания денег, финансирования терроризма или иных нарушений.

Подпись и дата	
Имя, Подпись	
Взам или №	
Подпись и дата	
Имя, №	

					ПП.09.02.07.491.19.26 ПЗ	Лист
						5
Изм.	Лист	№ докум.	Подпись	Дата		

Особенности:

Использует правила, аналитические модели и алгоритмы AI.

Обрабатывает большие объемы данных в реальном времени или батчем.

Включает оценку риска каждой операции.

3. Подозрительная операция

Определение: Это транзакция или серия операций, которые демонстрируют признаки потенциальных нарушений или преступлений по установленным правилам или моделям.

Признаки подозрительности:

Аномально большая сумма.

Необычный маршрут транзакции (например, перевод из страны с высоким риском).

Операции в короткий промежуток времени.

Связь с сомнительными контрагентами.

Использование подозрительных устройств или IP-адресов.

Несоответствие привычного поведения клиента.

Важность: Выявлять такие операции важно для предотвращения злоупотреблений и соблюдения требований регуляторов.

4. Правила и сценарии анализа

Определение: Это фиксированные или динамически формируемые наборы условий, по которым тестируется каждое транзакционное действие.

Типы правил:

Простые: сумма операции > установленного лимита.

Комплексные: серии операций за короткое время или определенные связки транзакций.

Географические: операции из неподдерживаемых или запрещенных стран.

Временные: операции, совершенные вне привычных временных интервалов.

Пример сценария:

«Если сумма перевода превышает 1 миллион рублей и перевод осуществляется из России в оффшорную юрисдикцию — отметить для проверки.»

5. Алгоритмы и модели оценки риска

Определение: Это инструменты анализа, построенные на статистике, машинном обучении или экспертных знаниях, которые позволяют определить уровень вероятности мошенничества или противоправной деятельности.

Категории:

Модели классификации: определяют, является ли операция подозрительной или нет, на основе обученных данных.

Подпись и дата	
Имя Подписи	
Взам или №	
Подпись и дата	
Имя № подл	

					ПП.09.02.07.491.19.26 ПЗ	Лист
						6
Изм.	Лист	№ докум.	Подпись	Дата		

Модели обнаружения аномалий: выявляют операции, отклоняющиеся от привычного поведения клиента или группы.

Классификация по уровню риска: присваивает транзакции степень опасности (низкий, средний, высокий риск).

Примеры техник:

Логистическая регрессия

Деревья решений

Нейронные сети

Методы кластеризации (например, K-средних)

6. Обнаружение аномалий

Определение: Процесс поиска отклонений от стандартных или ожидаемых Patterns поведения и транзакций.

Подходы:

Статистические: выявление транзакций с необычными параметрами относительно исторических данных.

Машинное обучение: автоэнкодеры, кластеризация, деревья случайного леса, методы градиентного бустинга.

Гибридные: сочетание правил и моделей машинного обучения для повышения точности.

Цель: минимизация ложных срабатываний и повышение точности выявления реальных рисков.

7. Клиент или держатель счета (Customer)

Определение: Физическое или юридическое лицо, являющееся владельцем счета или карты, совершающее транзакции.

Детали:

Личные данные: имя, дата рождения, паспортные данные.

Поведенческие характеристики: ежедневная активность, типичные суммы, география.

Исторические операции: типичные схемы поведения.

Роль: Анализ поведения клиента помогает выявлять необычные операции, которые могут указывать на мошенничество.

8. Структура клиентских сегментов

Определение: Разделение клиентов на группы по признакам поведения или характеристикам для повышения эффективности мониторинга.

Категории:

Потребители с низким уровнем транзакционной активности.

Потребители с высокой частотой операций.

Корпоративные клиенты.

Клиенты из определенных географических регионов.

Имя	Подпись и дата	Имя	Подпись	Дата	Изм.	Лист	№ докум.	Подпись	Дата	ПП.09.02.07.491.19.26 ПЗ	Лист
											7

Реализовать механизмы преобразования, нормализации и очистки данных (ETL-процессы).

Обеспечить постоянную актуальность информации.

2. Хранилище данных (Data Storage Layer)

Задачи:

Классический Data Warehouse для хранения структурированных данных.

Data Lake для хранения неструктурированных или полуструктурированных данных (логов, файлов, документов).

Резервное копирование и архивирование.

Функции:

Обеспечить быстрый доступ к историческим данным для аналитики.

Обеспечить безопасность и контроль доступа.

3. Обработчик данных (Data Processing Layer)

Технологии:

Потоковая обработка (Apache Kafka, Apache Flink, Spark Streaming).

Батч-обработка (Apache Spark, Hadoop).

Задачи:

Постоянное получение данных для анализа.

Обработка в реальном времени или партиями.

Предварительная фильтрация и агрегирование.

Особенности:

Обработка больших потоков данных без задержек.

Обеспечение отказоустойчивости и масштабируемости.

4. Аналитический движок (Analytics Engine)

Блоки:

Правила бизнес-логики: фиксированные сценарии и условия, по которым осуществляется первичная автоматическая проверка.

Модели машинного обучения: для оценки риска, выявления скрытых паттернов, обнаружения аномалий.

Модели обнаружения аномалий: выявление отклонений от стандартных моделей поведения.

Задачи:

Оценить каждую операцию по разным критериям.

Назначить уровни риска.

Автоматически пометить потенциально подозрительные операции.

5. Модуль принятия решений и реагирования

Функции:

Принятие автоматических решений (например, блокировка, отказ в транзакции).

Подпись и дата	
Имя подписи	
Взам или №	
Подпись и дата	
Имя № подписи	

					ПП.09.02.07.491.19.26 ПЗ	Лист
						9
Изм.	Лист	№ докум.	Подпись	Дата		

Уведомление клиентов или сотрудников банка.

Автоматические триггеры.

Система оповещений (SMS, Email, push-уведомления).

Задачи:

Позволяет проводить ручную проверку и корректировать решения.

Обеспечить управление правилами и моделями.

Простая навигация.

Визуализация данных и аналитики.

Возможность Быстрой реакции.

Задачи:

Хранение истории принятых решений.

Соблюдение требований законодательства и нормативных актов.

Передача данных регуляторам (например, отчетность по AML).

Автоматизированное создание отчетов.

Логирование и аудит операций.

Обеспечение прозрачности и ответственности.

Современные системы мониторинга используют комбинацию правил, статистических методов и машинного обучения для повышения эффективности обнаружения подозрительных операций.

1. Правила и сценарии (Rule-based analysis)

Это наиболее базовый и одновременно прозрачный способ анализа. Правила задаются экспертами — специалистами по рискам и вопросам compliance. Они позволяют быстро и понятно определить, какие операции являются явно подозрительными.

Операции, превышающие установленный лимит.

Переводы из стран с высоким уровнем риска или санкционных списков.

Операции в нерабочие часы.

Повторяющиеся транзакции с одними и теми же контрагентами.

Преимущество

Такой подход обеспечивает быстрое реагирование и абсолютную прозрачность решений. Правила легко понять и объяснить как внутри банка, так и регуляторам.

Ограничения

Правила не умеют выявлять сложные схемы, маскировочные операции и скрытую деятельность — они ограничены жестко прописанным сценарием.

2. Методы обнаружения аномалий

Почему это важно?

Банковские операции зачастую имеют сложный и разнообразный характер. Выявить признаки мошенничества или необычного поведения требует более тонкого анализа.

Основные подходы

2.1 Статистические методы

В основе — анализ отклонений от типичного поведения:

Z-score (стандартизация): операция, сумма которой отличается от среднего по клиенту или группе на несколько стандартных отклонений, считается подозрительной.

Межквартильный диапазон: операции, выходящие за границы межквартильного диапазона, автоматически вызывают подозрение.

Эти методы работают быстро и просто, их легко внедрить, однако требуют хорошей настройки пороговых значений.

2.2 Машинное обучение

Многие современные системы используют машинное обучение для более глубокого анализа поведения клиента и транзакций:

Кластеризация: группы операций делятся по схожести. Если новая операция сильно отличается, она считается потенциальной аномалией.

Классификация: обученные модели (например, случайный лес или градиентный бустинг) классифицируют операции как “подозрительные” или “безопасные” на основе исторических данных.

Почему это важно?

Машинное обучение позволяет выявлять сложные, скрытые закономерности, которые простые правила не замечают, и адаптироваться под новые схемы мошенничества.

3. Нейросетевые модели

Какие их преимущества?

Идиф. № докум.	Подпись и дата	Взам. или №	Имя Подписи	Подпись и дата						Лист
										11
Изм.	Лист	№ докум.	Подпись	Дата	ПП.09.02.07.491.19.26 ПЗ					

Нейросети способны анализировать последовательности данных, выявлять тонкие паттерны в поведении клиента.

Популярные модели:

Автоэнкодеры: обучаются только на “чистых” операциях. При возникновении подозрительной операции их способность восстанавливать исходные данные ухудшается, что сигнализирует о аномалии.

LSTM (долго-краткосрочные память): анализируют последовательности транзакций по времени, выявляя неожиданные изменения или переходы.

Почему это актуально?

Эти модели особенно полезны, когда нужно анализировать большое количество последовательных операций — например, поведение клиента за месяц или год.

4. Методы анализа изменений (Change Point Detection)

Что это?

Идея — выявлять резкие и внезапные изменения в поведении клиента или объеме транзакций — например, внезапное увеличение суммы переводов или частоты операций.

Применение:

Определение “точек смены” в поведении, что может свидетельствовать о компрометации аккаунта или мошеннической схеме.

Быстрое реагирование на необычные ситуации.

5. Графовые методы и связанный анализ

Что это?

Построение графов, где узлы — это клиенты или контрагенты, а связи — транзакции. Анализ таких структур помогает выявлять группы, связанные между собой подозрительными схемами.

Применение:

Обнаружение цепочек связанных мошеннических операций.

Визуализация связей для быстрого понимания ситуации.

Почему важно?

Мошенники часто используют сложные цепочки, чтобы скрыть свои действия. Графовые алгоритмы помогают их раскрывать.

6. Гибридные подходы

Что это?

Комбинация всех вышеперечисленных методов:

Сначала идет фильтрация по простым правилам.

Затем — углубленный анализ подозрительных случаев с помощью моделей машинного обучения.

Идентификация документа	Подпись и дата					Лист
	Имя Подписи					
	Взам или №					
	Подпись и дата					
Идентификация документа						Лист
	Изм.	Лист	№ докум.	Подпись	Дата	

в) Международные организации

FATF (Financial Action Task Force) — задает глобальные стандарты AML/CFT.

Basel Committee on Banking Supervision — регулирует устойчивость банков (тоже Basel III и др.).

ISO/IEC — стандарты по информационной безопасности и управлению рисками.

2. Ключевые регулятивные положения и требования

а) Законодательство РФ

Федеральный закон №115-ФЗ "О противодействии легализации доходов, полученных преступным путем, и финансированию терроризма" (ФЗ-115)

Ключевые пункты:

Обязательство идентификации клиента (KYC) — установление личности клиента при открытии счета.

Идентификация бенефициара — выявление реального владельца.

Формирование базы данных — текущие и архивные сведения о клиентах.

Мониторинг транзакций — автоматическое отслеживание аномальных или подозрительных операций.

Обязанность подачи отчёта о подозрительных операциях — в Росфинмониторинг в течение 3-х дней.

Обучение персонала — регулярное повышение квалификации.

б) Федеральные нормативы и руководства

Обязательные требования и рекомендации ЦБ РФ — по внутренним политикам, системам контроля, информационной безопасности, стресс-тестам, наличию системы управления рисками.

Национальный стандарт — обязательный для всех финансовых корпораций (например, требования к системе менеджмента информационной безопасности по ISO/IEC 27001).

3. Технические и организационные стандарты

а) AML/CFT стандартные процедуры

Проверка клиента ("Знай своего клиента" или KYC):

сбор удостоверяющих документов.

оценка источника дохода.

проверка клиента по черным спискам и санкционным базам.

Использование автоматических систем мониторинга:

системы, основанные на алгоритмах машинного обучения, правилах и сценариях.

автоматическая генерация меток, сигналов, предупреждений.

Подпись и дата	
Имя и подпись	
Взам или №	
Подпись и дата	
Имя и подпись	

					ПП.09.02.07.491.19.26 ПЗ	Лист
						15
Изм.	Лист	№ докум.	Подпись	Дата		

Постоянное обновление алгоритмов и данных обучения, чтобы системы не устаревали.

3. Надёжная защита и безопасность данных

Что это означает?

Все операции и данные — ценный актив. Защита личных данных клиентов и всей системы — это не только вопрос репутации, но и законодательных требований (например, GDPR или российский ФЗ-152).

Трудности:

Хакеры становятся всё изобретательнее, используют сложные атаки.

Внутренние угрозы — недобросовестные сотрудники, случайные утечки.

Требуются современные методы защиты, шифрование, многофакторная аутентификация.

Как решать:

Использование многоуровневых систем безопасности.

Внедрение технологий криптографии.

Постоянный мониторинг и аудит безопасности.

Обучение сотрудников отличает важность соблюдения правил информационной безопасности.

4. Интеграция разнородных систем и данных

Что это означает?

У каждой организации — свои программы, базы данных, платформы. Иногда данные хранятся в разных форматах, используют разные протоколы или старые системы. Объединить их в единое целое — чуть ли не самое трудное.

Почему это важно?

Если есть «слепые зоны» или необъединенные источники, опасность пропустить что-то важное или неправильно оценить ситуацию возрастает.

Что помогает:

Разработка унифицированных API.

Использование ETL-инструментов для интеграции данных.

Стандартизация форматов данных.

Внедрение платформ аналитики, объединяющих информацию из разных систем.

5. Быстрое реагирование на меняющиеся регуляции

Что это значит?

Финансовое законодательство — постоянно в движении. Требования к отчетности, борьбе с отмыванием денег, борьбе с мошенничеством меняются, и банкам нужно быстро подстраиваться.

Почему это сложно?

Постоянное обновление программного обеспечения и алгоритмов.

Подпись и дата	
Имя, Подпись	
Взам. или №	
Подпись и дата	
Имя, №, Подпись	

					ПП.09.02.07.491.19.26 ПЗ	Лист
						18
Изм.	Лист	№ докум.	Подпись	Дата		

Облачные платформы позволят гибко масштабировать системы, быстрее внедрять новые инструменты и снизить затраты на инфраструктуру.

6. Биометрия и другие инновации

Технологии распознавания лиц, отпечатков — всё это сделает идентификацию быстрее, удобнее и безопаснее.

1.2. Описание предмета автоматизации.

Обработка и анализ транзакций — это краеугольный камень современного банковского дела и финансовых технологий. Это процесс, в рамках которого миллионы операций проходят через системы банка, после чего они тщательно проверяются, классифицируются и анализируются с целью обеспечения безопасности, эффективности и соответствия нормативным требованиям. В условиях растущих объемов данных и усложнения схем мошенничества этот аспект становится особенно важен.

Почему правильная обработка транзакций критична для банка?

Обеспечение безопасности клиентов и банка: Быстрое выявление подозрительных операций позволяет предотвратить мошенничество и несанкционированный доступ.

Высокий уровень сервиса: Клиенты ценят мгновенное подтверждение своих операций — задержки вызывают недоверие и негативные отзывы.

Соответствие требованиям законодательства: Тщательный учет и возможность проведения аудита необходимы для соблюдения международных и местных нормативов.

Экономическая эффективность: Автоматизация и оптимизация обработки помогают снизить операционные издержки и повысить рентабельность.

Основные задачи при обработке и анализе транзакций

1. Реализация быстрой и корректной обработки операций

Обработка транзакций — это сердце работы любого банка или финансового учреждения. Она должна быть не только быстрой, но и безошибочной, ведь от этого напрямую зависит уровень доверия клиентов и репутация организации. В условиях современного рынка, когда клиенты ожидают мгновенного результата, а любые задержки вызывают недовольство, именно эффективность этого этапа становится ключевым фактором успеха.

Почему быстрота и точность так важны?

Быстрая обработка транзакций помогает укрепить доверие клиентов и повысить лояльность. Она снижает риски ошибок и злоупотреблений, что особенно важно в условиях увеличивающихся объемов данных. В условиях высокой конкуренции на рынке скоростные операции делают компанию более привлекательной для клиентов. Кроме того, аккуратность и скорость

Подпись и дата	
Имя, Подпись	
Взам. и Имя, Подпись	
Подпись и дата	
Имя, Подпись	

					ПП.09.02.07.491.19.26 ПЗ	Лист
						20
Изм.	Лист	№ докум.	Подпись	Дата		

формирования надежной репутации — залог устойчивого развития.

Основные компоненты успешной реализации

Ключевыми аспектами являются автоматизация процессов, использование надежной инфраструктуры и стандартизация процедур. Внедрение передовых систем автоматической обработки позволяет ускорить выполнение операций и снизить вероятность ошибок. Высокопроизводительные серверы, отказоустойчивые сети и системы резервного копирования обеспечивают стабильную работу и защищенность. Стандартизация алгоритмов и процедур гарантирует последовательность и качество выполнения. Проверка правильности данных и авторизации защищает от ошибок и злоупотреблений. Использование современных интерфейсов и протоколов обмена информацией, таких как API, обеспечивает скоростной и эффективный обмен данными между системами.

Технологические инструменты для обеспечения скорости и корректности

Решения, такие как Kafka или Spark Streaming, позволяют мгновенно реагировать на каждую операцию, обеспечивая обработку данных в реальном времени. In-memory базы данных типа Redis позволяют получать и сохранять информацию за миллисекунды. Машинное обучение помогает автоматически выявлять подозрительные операции, повышая безопасность. Стандартизованные API ускоряют интеграцию систем и обмен данными без задержек.

Как обеспечивается качество?

Перед обработкой данные проходят проверку и валидацию, что предотвращает ошибки. Многоуровневая авторизация подтверждает личность клиента и его права. Автоматический мониторинг и системы мгновенно выявляют подозрительные операции и блокируют их. Все действия фиксируются для целей аудита и постоянного совершенствования процессов.

2. Валидация и контроль данных

В цифровом мире каждое финансовое действие начинается с данных — вводимых клиентом или автоматически получаемых из систем. Ошибки или несоответствия в этих данных могут стать причиной задержек, неправильных операций или даже финансовых потерь. Поэтому вопрос валидации и контроля данных — это не просто технический момент, а жизненно важный элемент обеспечения надежности и безопасности всей системы.

Почему правильная проверка данных так важна?

Качественная валидация позволяет исключить ошибки на ранней стадии, снизить вероятность ошибок в расчетах и избежать возможных мошеннических действий. Она способствует тому, чтобы каждая транзакция соответствовала поставленным стандартам, что повышает доверие клиентов и

Подпись и дата						
Имя	Подпись					
Взам или №						
Подпись и дата						
Имя	Подпись					
Изм.	Лист	№ докум.	Подпись	Дата	ПП.09.02.07.491.19.26 ПЗ	Лист
						21

укрепляет репутацию банка. Ведь, как известно, доверие — основе бизнеса, а оно строится на аккуратности и честности.

Основные этапы проверки данных

Проверка полноты и формата: система автоматически проверяет, все ли обязательные поля заполнены и соответствуют ли они заданным форматам (например, правильный формат номера карты, даты, кода страны).

Валидация логики: мы убеждаемся, что введенные данные логически связаны между собой — например, сумма платежа не превышает установленных лимитов, а даты не противоречат реальному сроку.

Проверка корректности информации: используются внешние базы данных и сервисы для сверки данных, таких как банковские справочники, базы мошеннических схем или черные списки.

Подтверждение идентичности: внедрение многофакторной авторизации, биометрии или одноразовых кодов помогает удостовериться, что инициатор транзакции — действительно тот, за кого себя выдает.

Аналитика и автоматические проверки: с помощью алгоритмов машинного обучения и аналитики система выявляет отклонения от привычных моделей поведения, что позволяет своевременно заметить подозрительные операции.

Какие инструменты используются для контроля?

Использование современных программных решений и автоматизированных систем существенно повышает эффективность. Это API-валидаторы, встроенные проверки в интерфейсе, системы автоматического анализа данных и базы данных с постоянно обновляемой информацией о мошенничестве и рисках.

Также важна интеграция с внешними источниками — например, международными и национальными базами проверки клиентов на предмет санкций или мошеннических схем. Всё это помогает обеспечить, что каждый запрос проходит многоступенчатую проверку и соответствует строгим стандартам.

Почему это важно для бизнеса и клиентов

Обеспечение безошибочной работы системы: сокращение количества неправильных транзакций и ошибок, что снижает операционный риск и издержки.

Защита от мошенничества: автоматическая проверка и аналитика помогают своевременно выявлять подозрительные операции, предотвращая финансовые потери и репутационные риски.

Уверенность клиентов: прозрачная и безопасная обработка данных формирует доверие и повышает уровень удовлетворенности.

Подпись и дата	
Имя Подписи	
Взам или №	
Подпись и дата	
Имя № подписи	

					ПП.09.02.07.491.19.26 ПЗ	Лист 22
Изм.	Лист	№ докум.	Подпись	Дата		

Соответствие нормативам: строгий контроль данных помогает соблюдать законы и регуляторные требования, минимизируя риски штрафов и санкций.

3. Мониторинг подозрительных и аномальных операций

В мире финансовых услуг безопасность — это неотъемлемая часть качественного обслуживания клиентов. Даже самые устойчивые системы могут стать мишенью для мошенников или столкнуться с ошибками. Чтобы избежать крупных потерь, репутационных рисков и обеспечить доверие, важно постоянно следить за каждой транзакцией и вовремя реагировать на подозрительные ситуации.

Почему мониторинг аномалий так важен?

Профилактика мошенничества: своевременное выявление мошеннических схем позволяет остановить преступника до нанесения серьезного урона.

Защита клиентов: снижение риска потери средств или других негативных последствий для клиентов и их данных.

Соответствие нормативам: многие регуляторы требуют постоянного контроля за операциями для предотвращения легализации доходов и финансирования терроризма.

Поддержка репутации: клиенты ценят безопасность своих данных и средств, зона высокой ответственности — одна из основ доверия к вашему бизнесу.

Как работает механизм обнаружения аномалий?

Многоуровневый анализ: система объединяет правила, основанные на шаблонах поведения, и машинное обучение для выявления неожиданных или необычных операций.

Настройка порогов и правил: задаются параметры, по которым транзакция считается потенциально подозрительной — например, сумма, превышающая стандартные лимиты, необычные географические направления или попытки обхода систем безопасности.

Аналитика в реальном времени: системы отслеживают операции в режиме онлайн, мгновенно фиксируют любые отклонения от нормы и отправляют уведомления.

Исторический анализ: анализируются последовательности операций для выявления сложных схем мошенничества, которые не проявляются при статическом контроле.

Какие инструменты и технологии применяются?

Машинное обучение и искусственный интеллект: позволяют выявлять новые, неучтённые ранее схемы мошенничества, постоянно обучаясь и адаптируясь к новым угрозам.

Поведенческий анализ: системы анализируют привычки пользователя, их изменения или необычные модели поведения.

Подпись и дата						
Имя Подписи						
Взам или №						
Подпись и дата						
Имя № подл						
Изм.	Лист	№ докум.	Подпись	Дата	ПП.09.02.07.491.19.26 ПЗ	Лист
						23

Модели ИИ анализируют поведение клиента и транзакции, выявляя нестандартные или опасные паттерны. Такой подход постоянно совершенствуется и адаптируется к новым схемам мошенничества.

3. Автоматизация обработки (RPA)

Роботы помогают автоматизировать рутинные задачи — проверку данных, формирование отчетов, уведомления — что снижает риск ошибок и ускоряет обработку.

4. Визуализация данных и аналитические панели

Интерактивные дашборды позволяют аналитикам и сотрудникам службы безопасности получать информацию о текущем статусе операций и быстро принимать управленческие решения.

Реальные сценарии и практические примеры

Представьте, что клиент обычно совершает небольшие транзакции внутри страны. Внезапно появляется крупный перевод за границу, что значительно отклоняется от привычных моделей.

Автоматическая система мгновенно обнаруживает отклонение — например, с помощью ИИ или правил — и инициирует определённые действия: задержку операции, отправку уведомления клиенту, запрос дополнительной проверки. Такой подход позволяет не только оперативно блокировать потенциальное мошенничество, но и сохранить доверие клиента за счет быстрого реагирования.

1.3. Анализ существующих разработок.

1. Обзор предметной области

Мониторинг банковских операций представляет собой комплексную задачу, направленную на обеспечение безопасности финансовых транзакций, выявление подозрительной деятельности и предотвращение мошенничества. В современных условиях банковские учреждения обрабатывают миллионы транзакций ежедневно, что обуславливает необходимость применения автоматизированных систем мониторинга.

Системы мониторинга банковских операций предназначены для решения следующих основных задач: выявление аномальных и подозрительных транзакций в режиме реального времени; анализ поведенческих паттернов клиентов; предотвращение несанкционированного доступа к счетам; выполнение требований законодательства в области противодействия отмыванию денег; формирование отчетности для регулирующих органов.

2. Анализ существующих решений

2.1. Зарубежные системы мониторинга

SAS Fraud Management

Инф. № докум.	Подпись и дата	Взам. инв. №	Имя, Подпись	Подпись и дата						
Изм.	Лист	№ докум.	Подпись	Дата	ПП.09.02.07.491.19.26 ПЗ					Лист
										25

К преимуществам системы относятся высокая производительность и масштабируемость, развитые аналитические инструменты, широкая интеграция с банковскими системами. При этом следует отметить высокую стоимость лицензий и внедрения, сложность настройки и адаптации, значительные требования к вычислительным ресурсам.

Система мониторинга мошенничества, разработанная компанией FICO, получила широкое распространение среди крупных банков по всему миру. К основным функциональным возможностям относятся использование нейросетевых моделей для обнаружения мошенничества, анализ в реальном времени, адаптивные модели, обучающиеся на новых данных, многоуровневая система защиты.

Oracle Financial Services Analytical Applications

2.2. Отечественные системы мониторинга

Система разработана Банком России для координации реагирования на компьютерные атаки в кредитно-финансовой сфере. Функциональные возможности включают сбор и анализ данных о инцидентах информационной безопасности, обмен данными между участниками финансового рынка,

Преимуществами являются государственная поддержка и координация, единая точка контакта для банковского сообщества, актуальная информация о текущих угрозах. К недостаткам следует отнести ориентацию на информационную безопасность, а не на операционный мониторинг, ограниченные возможности для отдельных банков, преимущественно реактивный характер работы.

Система мониторинга событий безопасности адаптирована для банковского сектора. Основные функциональные возможности включают мониторинг событий в режиме реального времени, корреляционный анализ, автоматическое реагирование на инциденты, визуализацию и отчётность.

«Аурелия» (РТКЦ «Центр»)

К преимуществам относятся отечественная разработка, специализация на банковских операциях, адаптация к российскому законодательству. Недостатки выражаются в ограниченных возможностях машинного обучения, меньшей масштабируемости по сравнению с зарубежными аналогами.

При проведении сравнительного анализа рассмотренных систем мониторинга банковских операций необходимо оценить их по ключевым параметрам.

По показателю стоимости внедрения зарубежные решения SAS Fraud Management, FICO Falcon и Oracle OFSAA характеризуются высоким уровнем затрат. Отечественные разработки в лице FinCERT, SafeBoard и «Аурелия» демонстрируют значительно более доступную структуру расходов на внедрение.

Показатель масштабируемости наиболее высоким значениям соответствует у зарубежных корпоративных систем, тогда как отечественные решения обеспечивают средний уровень данного параметра.

В части применения технологий машинного обучения и искусственного интеллекта зарубежные системы SAS Fraud Management и FICO Falcon демонстрируют развитые возможности. Решения Oracle OFSAA, SafeBoard обеспечивают средний уровень данных технологий. Системы FinCERT и «Аурелия» имеют ограниченные возможности в данной области.

Соответствие требованиям Федерального закона № 115-ФЗ обеспечивается всеми рассмотренными системами без исключения.

Показатель локализации полностью реализован в отечественных разработках, тогда как зарубежные системы требуют дополнительной настройки для обеспечения соответствия российским нормативным требованиям.

Техническая поддержка корпоративного уровня доступна для зарубежных решений. Оперативность технической поддержки отечественных систем существенно ниже.

Сроки внедрения зарубежных решений составляют от шести до восемнадцати месяцев в зависимости от масштаба системы. Оперативность внедрения отечественных разработок значительно превышает аналогичные показатели зарубежных систем.

Целевая аудитория зарубежных решений преимущественно включает крупные банки. Отечественные системы ориентированы на средние и малые кредитные организации.

4. Анализ функциональных возможностей

4.1. Типовые функции современных систем мониторинга

На основании проведённого анализа существующих разработок выделены следующие типовые функциональные компоненты систем мониторинга банковских операций.

Модуль сбора данных обеспечивает получение транзакционных данных из автоматизированных банковских систем, интеграцию с процессинговыми центрами, сбор данных из внешних источников, нормализацию и преобразование данных.

Модуль анализа выполняет применение статических правил и пороговых значений, анализ поведенческих паттернов, статистическое моделирование, машинное обучение и предиктивную аналитику, сетевой анализ связей.

Модуль выявления осуществляет детектирование подозрительных операций, формирование алертов и уведомлений, приоритизацию инцидентов, расчёт степени риска.

Модуль расследования предоставляет инструменты для аналитиков, формирует кейсы расследования, обеспечивает документирование результатов, управляет workflow инцидентов.

Подпись и дата	
Имя	Подпись
Взам или №	
Подпись и дата	
Имя	Подпись

					ПП.09.02.07.491.19.26 ПЗ	Лист
						28
Изм.	Лист	№ докум.	Подпись	Дата		

Модуль отчётности формирует операционные отчёты, обеспечивает подготовку регуляторной отчётности, предоставляет аналитические дашборды, автоматизирует генерацию заключений.

4.2. Технологический стек современных систем

Анализ технологий, применяемых в современных системах мониторинга, показывает преобладание реляционных систем управления базами данных для хранения транзакционных данных. Технологии NoSQL применяются для хранения профилей и метаданных. Обработка потоковых данных с использованием специализированных инструментов обеспечивает анализ в реальном времени. Среды машинного обучения применяются для выявления аномалий. Графовые базы данных обеспечивают сетевой анализ. Контейнеризация используется для развёртывания и масштабирования компонентов системы.

5. Выявленные недостатки существующих решений

По результатам анализа существующих разработок выделены следующие проблемы и ограничения.

Высокая стоимость владения коммерческими решениями от зарубежных вендоров обуславливает значительные инвестиции, что делает подобные системы недоступными для небольших банков и кредитных организаций.

Сложность интеграции большинства существующих систем, представляющих собой монолитные решения, требует трудоёмкой интеграции с существующими автоматизированными банковскими системами и процессинговыми платформами.

Значительное число ложных срабатываний систем на основе статических правил генерирует большое количество алертов, требующих ручной проверки, что создаёт существенную нагрузку на аналитиков.

Ограниченная адаптивность многих систем не позволяет оперативно адаптироваться к новым схемам мошенничества без длительной перенастройки.

Отсутствие единого стандарта, когда различные разработчики применяют собственные форматы данных и протоколы интеграции, затрудняет обмен информацией между системами.

Недостаточная автоматизация, при которой значительная часть процессов расследования и принятия решений остаётся ручной, замедляет реагирование на инциденты.

6. Обоснование необходимости разработки

На основании проведённого анализа существующих разработок сформулированы следующие выводы.

Подпись и дата	
Имя	Подпись
Взам или №	
Подпись и дата	
Имя	Подпись

					ПП.09.02.07.491.19.26 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		29

Наличие рыночной ниши обусловлено тем, что существующие отечественные решения либо ориентированы на крупные банки, либо представляют собой системы информационной безопасности общего назначения, не учитывающие специфику финансового мониторинга.

Потребность в доступном решении для средних и небольших банков, предполагающем оптимальное соотношение функциональности и стоимости, подтверждает актуальность разработки.

Необходимость применения современного подхода с использованием микросервисной архитектуры, потоковой обработки данных и современных алгоритмов машинного обучения позволит создать более гибкое и масштабируемое решение.

Требования регулятора, выражающиеся в ужесточении требований Банка России к финансовому мониторингу, формируют устойчивый спрос на системы данного класса.

Таким образом, разработка новой системы мониторинга банковских операций с современной архитектурой, оптимальным функциональным набором и доступной стоимостью внедрения является актуальной и востребованной задачей, соответствующей потребностям современного банковского сектора.

1.4. Используемое программное обеспечение.

В ходе преддипломной практики по теме "Мониторинг банковских операций" было применено множество инструментов и технологий, каждый из которых сыграл ключевую роль в реализации проекта. В данном разделе подробно рассматриваются используемые языки программирования, системы управления базами данных, инструменты для анализа и визуализации данных, а также программные средства для разработки.

1. Языки программирования

1.1 Python

Python — это высокоуровневый язык программирования, который был выбран благодаря своей простоте и мощной экосистеме библиотек, что делает его особенно подходящим для задач анализа данных и автоматизации процессов. В рамках проекта Python использовался для:

- Разработки скриптов для автоматической обработки данных: с помощью Python были созданы скрипты, которые автоматически загружают данные о банковских операциях, обрабатывают их и готовят для анализа.
- Внедрения алгоритмов машинного обучения: библиотека Scikit-learn позволила разработать модели, которые помогают выявить аномалии в банковских транзакциях, что имеет важное значение для предотвращения мошенничества.

Имя № папки	Подпись и дата				
	Имя Набл				
	Взам или №				
	Подпись и дата				
В ходе преддипломной практики по теме "Мониторинг банковских операций" было применено множество инструментов и технологий, каждый из которых сыграл ключевую роль в реализации проекта. В данном разделе подробно рассматриваются используемые языки программирования, системы управления базами данных, инструменты для анализа и визуализации данных, а также программные средства для разработки. 1. Языки программирования 1.1 Python Python — это высокоуровневый язык программирования, который был выбран благодаря своей простоте и мощной экосистеме библиотек, что делает его особенно подходящим для задач анализа данных и автоматизации процессов. В рамках проекта Python использовался для: - Разработки скриптов для автоматической обработки данных: с помощью Python были созданы скрипты, которые автоматически загружают данные о банковских операциях, обрабатывают их и готовят для анализа. - Внедрения алгоритмов машинного обучения: библиотека Scikit-learn позволила разработать модели, которые помогают выявить аномалии в банковских транзакциях, что имеет важное значение для предотвращения мошенничества.					
					Лист
ПП.09.02.07.491.19.26 ПЗ					30
Изм.	Лист	№ докум.	Подпись	Дата	

- Удобные средства для загрузки, обработки и анализа данных, включая фильтрацию и агрегацию.
- Возможность манипуляции данными в табличной форме, что позволяет разрабатывать сложные аналитические отчеты и визуализации.
- Поддержку работы с временными рядами, что критично для банковского мониторинга.

3.2 NumPy

NumPy — это библиотека для научных расчётов в Python, которая использовалась для:

- Проведения численных расчетов, требующих высокой производительности.
- Обработки многомерных массивов данных, что удобно для работы с большими объемами информации и выполнения математических операций.

3.3 Matplotlib и Seaborn

Эти библиотеки были использованы для визуализации данных:

- Matplotlib — обеспечила создание графиков и диаграмм для представления результатов анализа, что помогает лучше понять тенденции и аномалии.
- Seaborn — предоставляет удобные функции для создания более сложных и красивых визуализаций, которые помогают в интерпретации

1.5. Описание используемых языков программирования.

Python - высокоуровневый язык программирования общего назначения, известный своей простотой, читаемостью кода и универсальностью. Он широко используется для веб-разработки (Django, Flask), анализа данных, искусственного интеллекта и машинного обучения.

Javascript — это язык программирования высокого уровня, который используется для создания интерактивных веб-страниц, клиентских и серверных приложений. Он является стандартом для браузеров, позволяя управлять контентом, анимацией и обрабатывать действия пользователей.

SQL - это стандартный язык программирования, используемый для взаимодействия с реляционными базами данных. Он позволяет создавать, читать, обновлять и удалять данные (операции CRUD), а также управлять структурой БД (таблицами). SQL работает в таких системах, как PostgreSQL, MySQL, Oracle и MS SQL.

Django — это популярный бесплатный веб-фреймворк с открытым исходным кодом, написанный на языке Python. Он предназначен для быстрой разработки безопасных и масштабируемых сайтов, предоставляя готовые инструменты для работы с базами данных, аутентификации пользователей и управления контентом.

Имя	Подпись и дата
Имя	Подпись
Взам	Имя
Имя	Подпись и дата
Имя	Подпись

					ПП.09.02.07.491.19.26 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		32

2. ПРАКТИЧЕСКАЯ ЧАСТЬ.

2.1. Постановка задачи.

Цель практики:

Создать систему мониторинга банковских операций, которая поможет автоматически отслеживать и анализировать транзакции, выявлять потенциально подозрительные и мошеннические действия, чтобы повысить уровень безопасности и надежности банковских процессов.

Задачи практики:

- Ознакомиться с требованиями и особенностями системы мониторинга в банковской сфере.
- Изучить современные методы и алгоритмы для обнаружения аномальных транзакций.
- Разработать архитектуру системы, включающую сбор, хранение и анализ данных о банковских операциях.
- Реализовать программное решение для автоматической обработки и анализа транзакций.
- Внедрить алгоритмы для выявления подозрительных операций и формирования тревожных уведомлений.
- Создать удобный интерфейс для отображения статистики и ключевых событий системы.
- Провести тестирование созданной системы и подготовить документацию по ее использованию.

Обоснование выбора задачи:

Реализация системы мониторинга является важной задачей для современной банковской деятельности, поскольку помогает минимизировать риски мошенничества и обеспечивает дополнительный уровень контроля. Практическая работа по созданию такой системы позволит укрепить профессиональные навыки и получить ценное понимание процессов автоматизации контроля безопасности.

2.2. Выбор сред разработки.

Для реализации проекта по созданию системы мониторинга банковских операций выбран интегрированный механизм разработки PyCharm в качестве основной среды работы на языке программирования Python.

Подпись и дата	
Имя Подписи	
Взам или №	
Подпись и дата	
Имя № подписи	

					ПП.09.02.07.491.19.26 ПЗ	Лист
						33
Изм.	Лист	№ докум.	Подпись	Дата		

Обоснование выбора:

PyCharm предоставляет мощные инструменты для написания, тестирования и отладки кода, что значительно повышает продуктивность и качество разработки.

Встроенная поддержка Python, системы контроля версий, управление виртуальными окружениями и автоматическое оформление кода делают рабочий процесс удобным и организованным.

Возможность интеграции со сторонними библиотеками и инструментами для анализа данных и машинного обучения облегчает выполнение различных этапов проекта.

Удобный интерфейс и богатый функционал позволяют быстро ориентироваться в проекте и эффективно управлять его структурой. Использование PyCharm способствует четкому структурированию разработки, снижает риск ошибок и позволяет сосредоточиться на решении поставленных задач, что важно для достижения успеха в реализации системы мониторинга.

2.3.Вайрфрейм программного продукта.

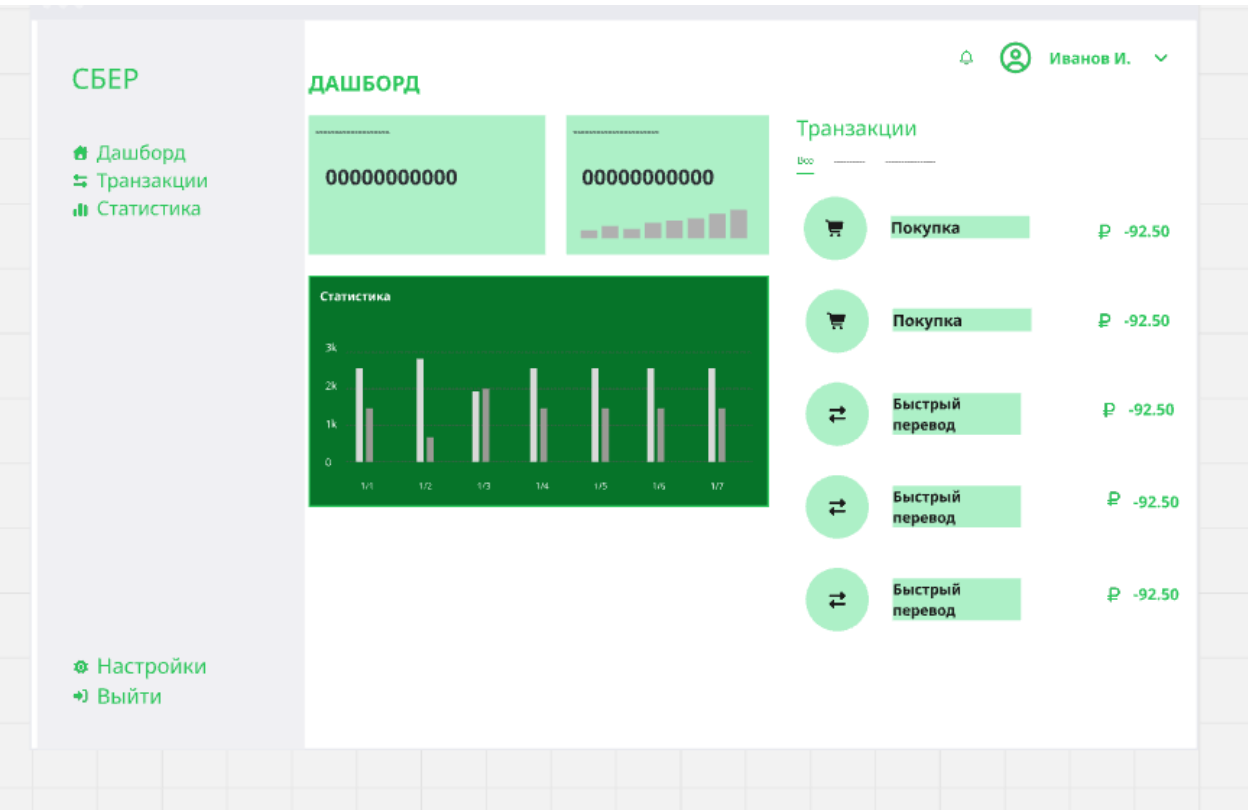


Рисунок 2.1 – «Вайрфрейм главной страницы».

Подпись и дата	
Имя Подписи	
Взам или №	
Подпись и дата	
Имя № подл	

2.4. Структура программного продукта.

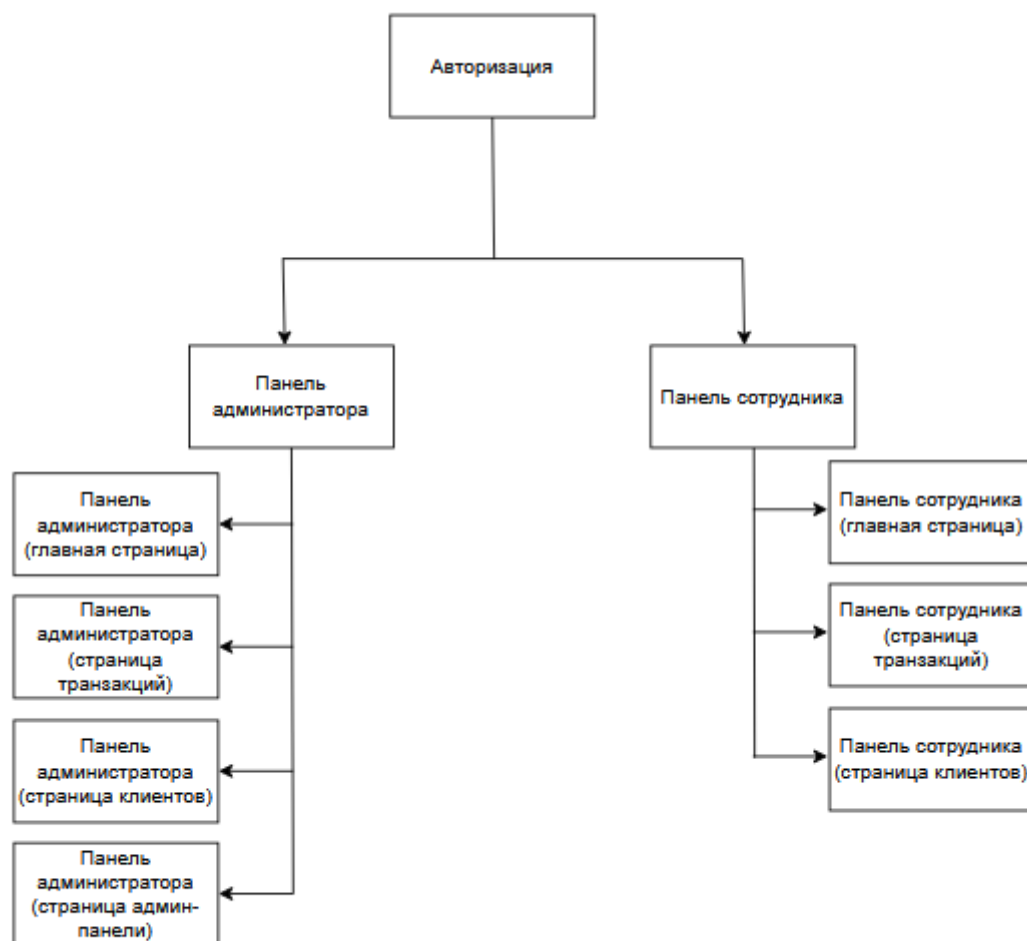


Рисунок 2.2 – «Структура программного продукта».

Основой системы является модульная архитектура, которая обеспечивает надежность, расширяемость и удобство в эксплуатации. Основные компоненты системы включают:

1. Модуль сбора данных.

Отвечает за автоматический импорт транзакций из различных источников (банковских систем, баз данных).

Поддерживает интеграцию с API банковских систем для получения актуальных данных в режиме реального времени.

2. Модель хранения данных.

Обеспечивает структурированное хранение всей информации о транзакциях, пользователях и правилах мониторинга.

Реализована с использованием базы данных что обеспечивает надежность и масштабируемость.

3. Модуль анализа и обнаружения аномалий.

Подпись и дата

Имя Подпись

Взам или №

Подпись и дата

Имя № подл

4. Реализация основных модулей.

Создание модуля сбора и хранения данных.

Разработка модуля анализа транзакций и выявления подозрительных операций.

Разработка интерфейса для просмотра и управления транзакциями.

Реализация модуля отчетности и системы настройки.

5. Тестирование и отладка.

Проведение модульных и интеграционных тестов для выявления ошибок.

Проверка производительности и надежности системы.

Исправление выявленных замечаний и оптимизация кода.

6. Внедрение и эксплуатация.

Развертывание системы в рабочей среде.

Обучение пользователей работе с системой.

Организация поддержки и регулярных обновлений.

7. Мониторинг и развитие.

Постоянное отслеживание работы системы.

Внедрение новых функций по мере появления новых требований.

Обеспечение безопасности и актуальности системы.

2.6. Модели бизнес-процессов.

1. Получение и первичная обработка данных о транзакциях.

Обеспечиваем бесперебойный поток сведений о всех операциях, поступающих из разных источников, будь то банковские системы или платёжные платформы.

Способ реализации:

Автоматизированно получаем данные о каждой транзакции.

Проверяем их на соответствие стандартам: чтобы информация была точной и полноценной.

Надёжно сохраняем в системе для дальнейшего анализа и отчетности.

Так мы создаем надежную базу для последующей работы системы и гарантируем, что все операции доступны для анализа в любой момент.

2. Аналитика и выявление рисков операций.

Это профессиональный этап — мы изучаем каждую транзакцию с целью найти возможные опасности или мошеннические схемы.

Способ реализации:

Запускаем специальные алгоритмы и правила, настроенные на обнаружение необычных моделей.

Каждая операция получает статус — «безопасно» или «подозрительно».

Подпись и дата

Имя Подписи

Взам или №

Подпись и дата

Инд. № подл

Для подозрительных случаев система автоматически формирует уведомления, чтобы оперативно привлечь внимание специалистов.

Благодаря автоматизированной аналитике мы не пропустим ни одной потенциально опасной транзакции и сможем быстро реагировать на угрозы.

3. Ручная сверка и принятие решений по подозрительным операциям.

На этом этапе наши сотрудники вмешиваются — проверяют все нюансы подозрительных транзакций.

Обзоры проводят опытные аналитики, разбираясь в деталях каждой операции.

При необходимости связываются с клиентами или проводят дополнительные проверки.

В итоге принимают решение: заблокировать транзакцию, оставить как есть или считать её безрисковой.

Это сбалансированный контроль — помогает минимизировать ошибки и одновременно обеспечить высокий уровень защиты клиентов и банка.

4. Подготовка и доставка отчетов.

Регулярно формируем отчеты — для руководства и регуляторов, чтобы они были всегда в курсе актуальных рисков.

Интеллектуально агрегируем сведения о подозрительных операциях, общем числе транзакций и ключевых трендах.

Быстро отправляем готовые отчеты по расписанию или по запросу.

В случае критичных событий — незамедлительно уведомляем руководство и сотрудников.

Обеспечиваем прозрачность и прозрачную коммуникацию — важнейшие элементы надежной системы управления рисками.

5. Настройка, сопровождение и развитие системы.

Поддерживаем эффективность системы на постоянной основе — чтобы она всегда оставалась актуальной и надежной.

Регулярно внедряем новые правила и алгоритмы анализа, учитывая динамику мошеннических схем.

Обновляем настройки и параметры по мере необходимости.

Обучаем команду, чтобы они могли максимально эффективно работать с системой.

Постоянное совершенствование делает систему устойчивой к новым вызовам и обеспечивает долгосрочную безопасность.

Подпись и дата	
Имя, Подпись	
Взам. Имя, №	
Подпись и дата	
Имя, №	

					ПП.09.02.07.491.19.26 ПЗ	Лист 38
Изм.	Лист	№ докум.	Подпись	Дата		

2.7. Функциональная диаграмма IDEF0

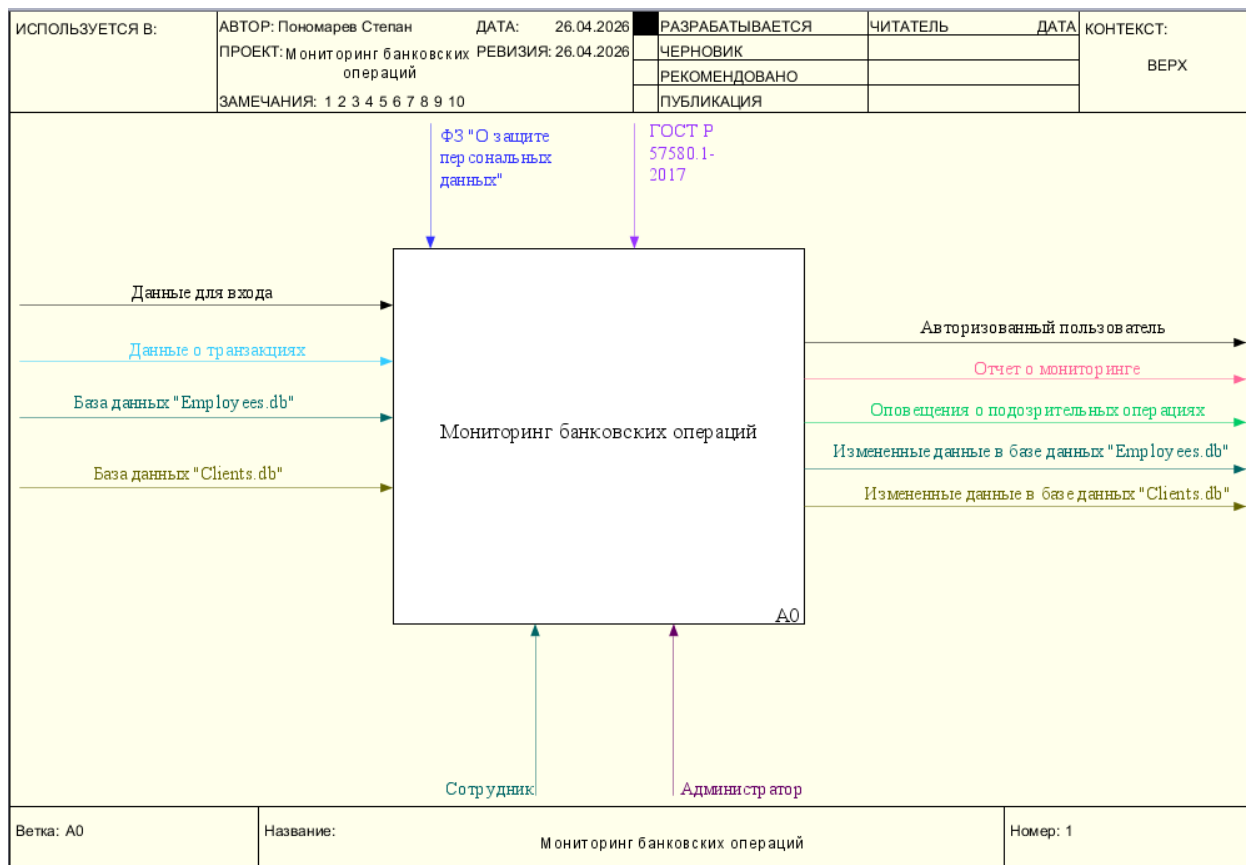


Рисунок 2.3 – «Диаграмма IDEF0 слой А-0».

Идиф. №	Подписчик и дата	Взам или №	Имя подписч	Подписчик и дата

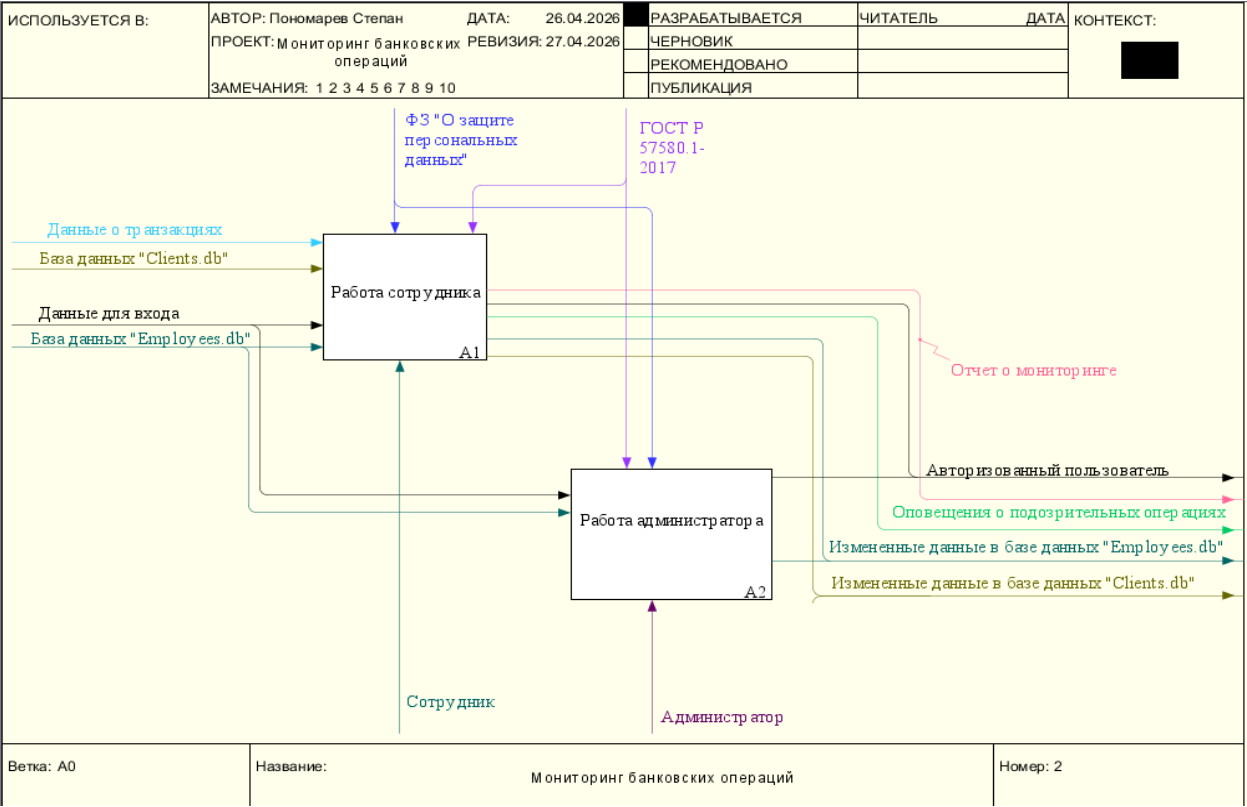


Рисунок 2.4 – «Диаграмма IDEF0 слой A0».

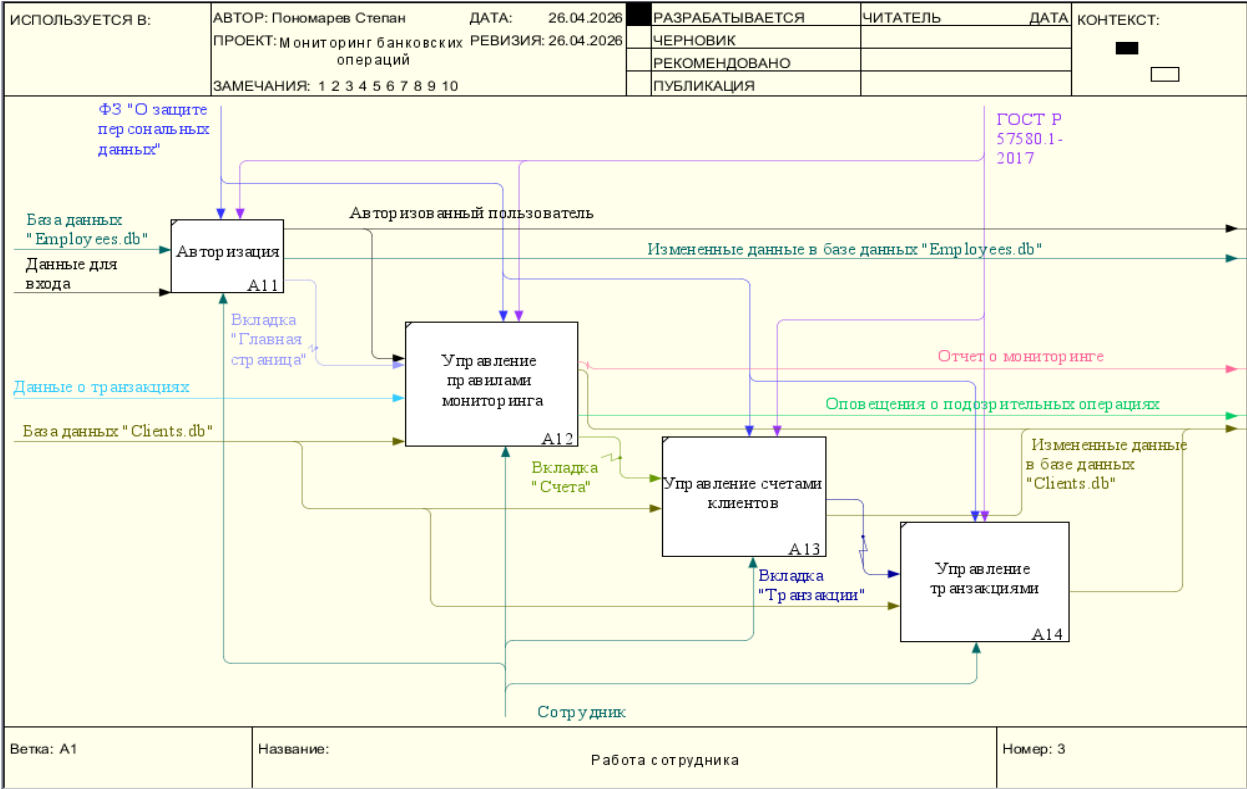


Рисунок 2.5 – «Диаграмма IDEF0 декомпозиция работы сотрудника».

Подпись и дата	
Имя Подписи	
Время	
Подпись и дата	
Имя Подписи	

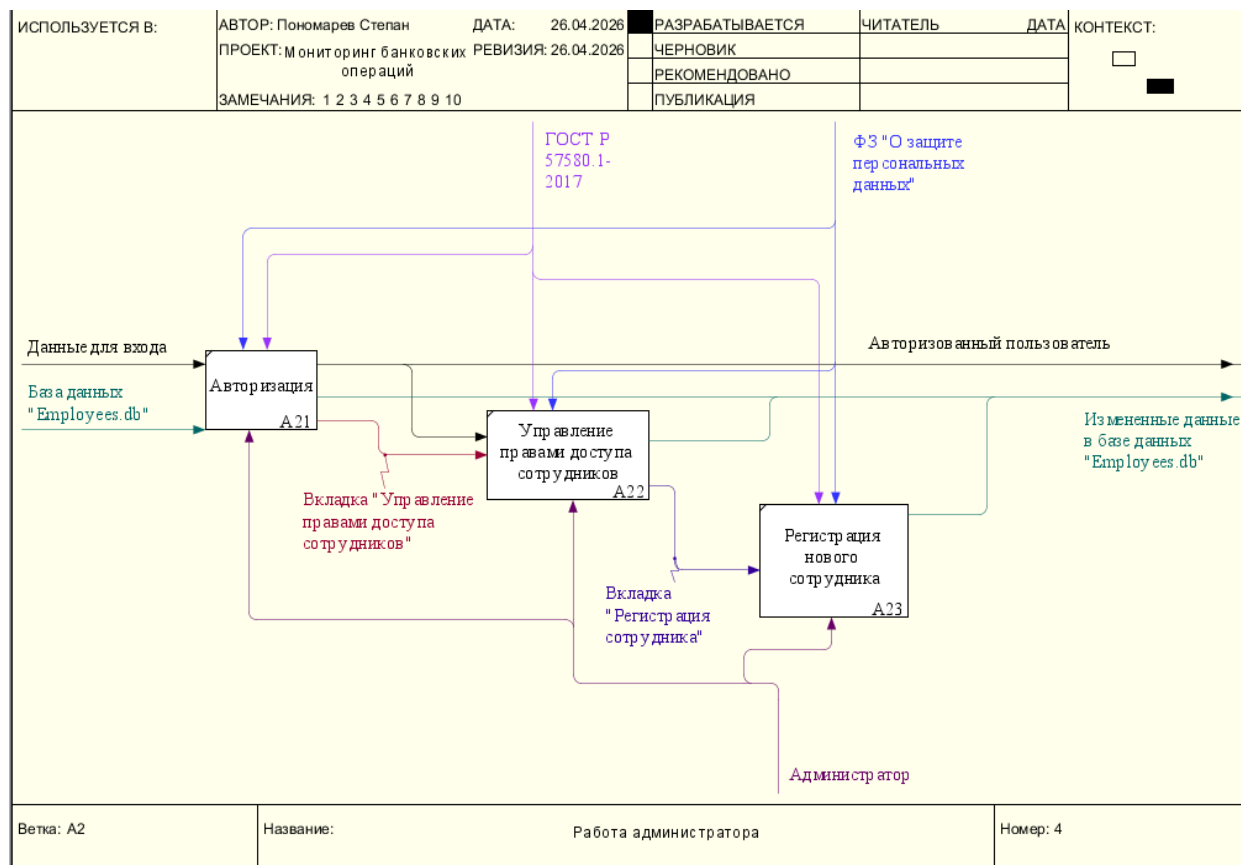


Рисунок 2.6 – «Диаграмма IDEF0 декомпозиция работы администратора».

2.8. Диаграмма потоков данных DFD

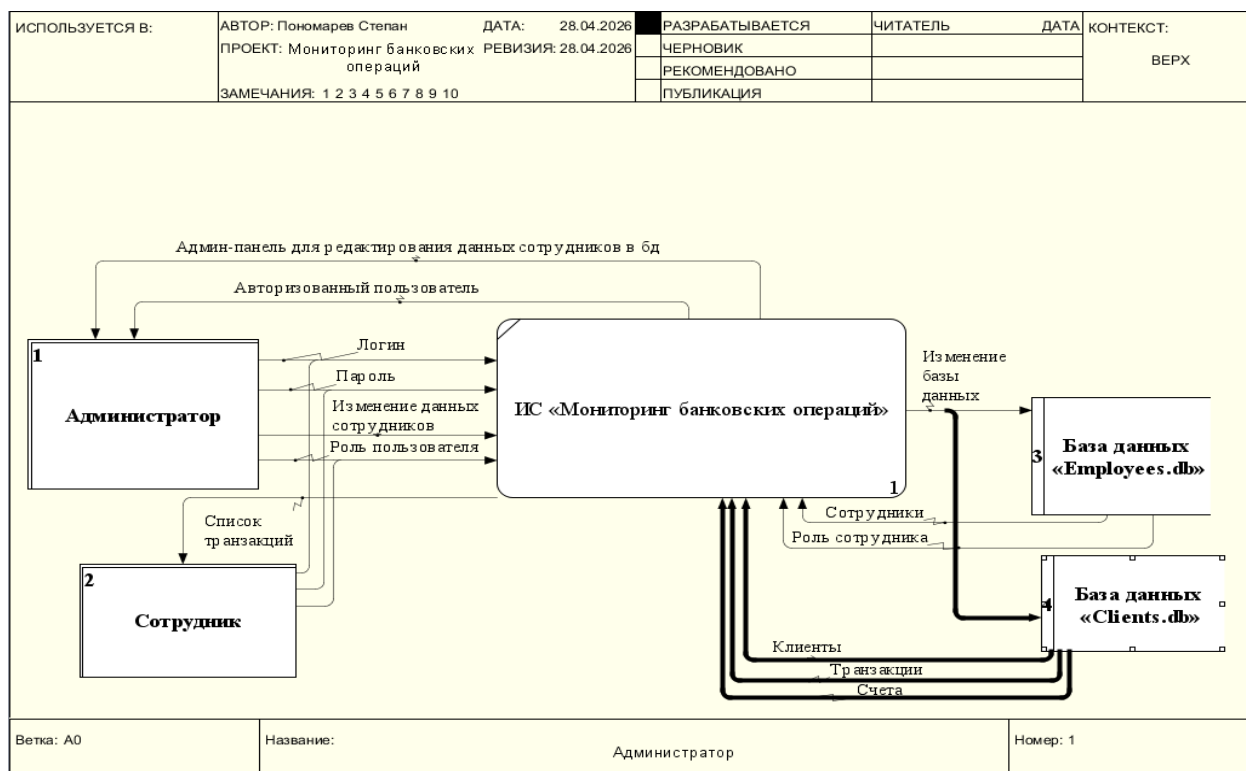


Рисунок 2.7 – «Диаграмма DFD ветка A0».

Подпись и дата	
Имя Подписи	
Взам или №	
Подпись и дата	
Имя № подписи	

Изм.	Лист	№ докум.	Подпись	Дата

ПП.09.02.07.491.19.26 ПЗ

2.9. Диаграмма «сущность-связь».

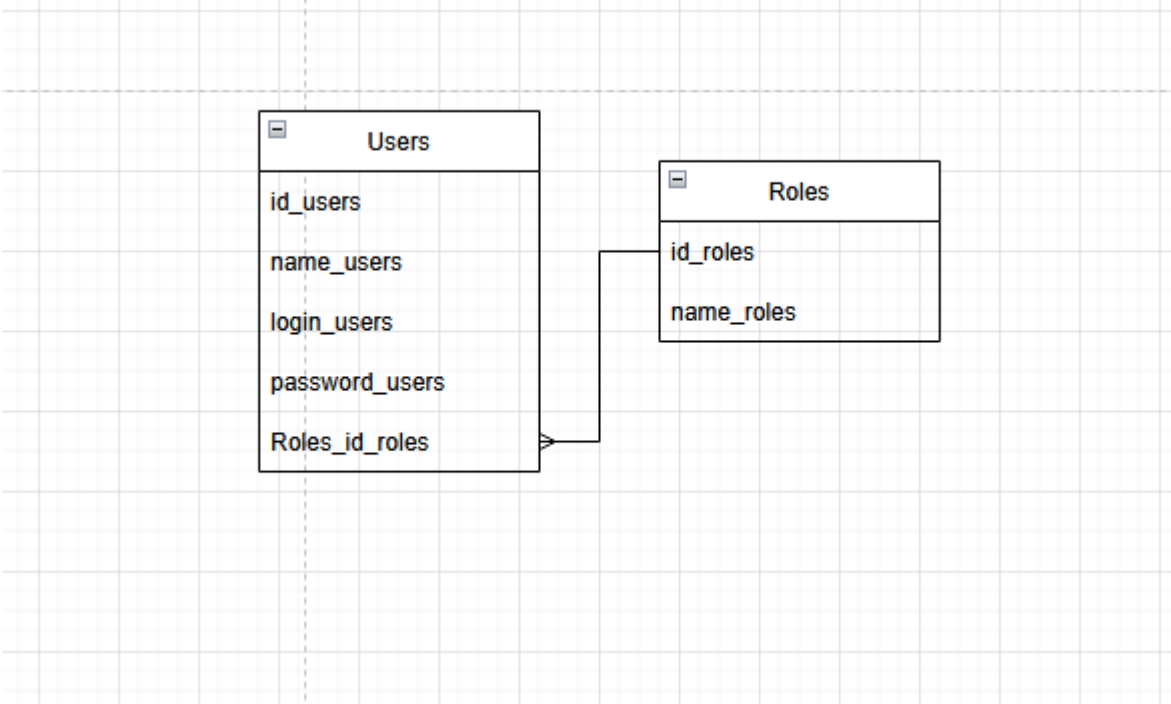


Рисунок 2.8 – «Сущность-связь базы данных Users».

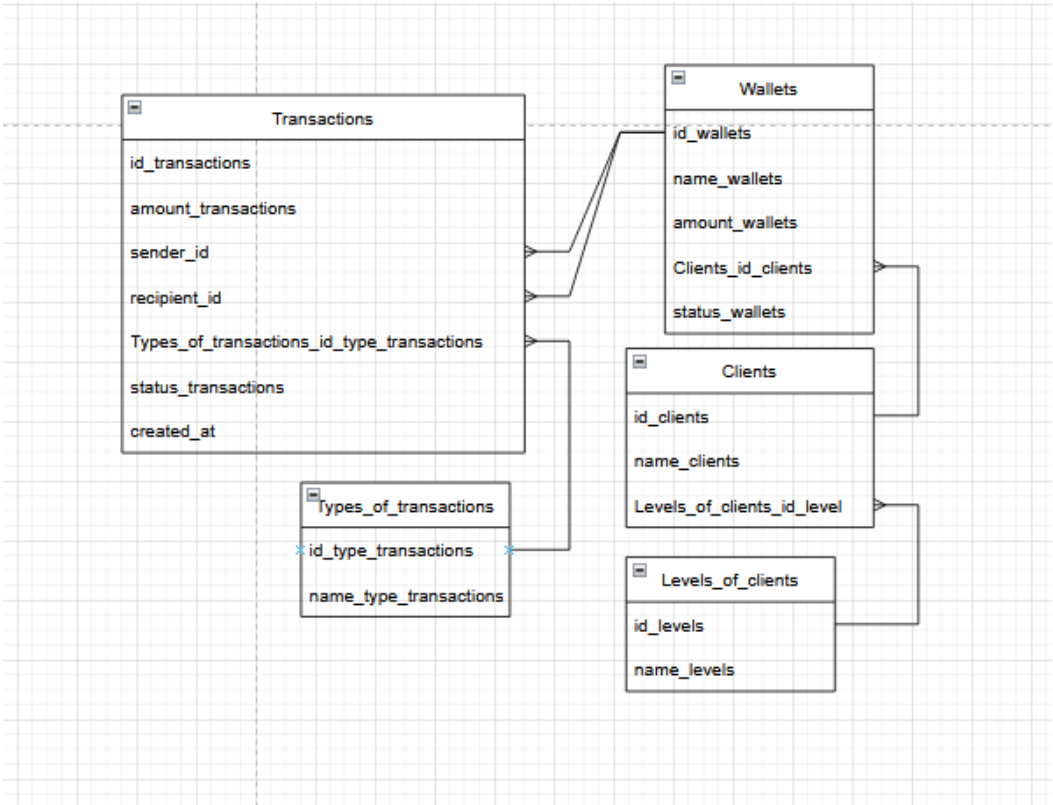


Рисунок 2.9 – «Сущность-связь базы данных Clients».

Подпись и дата	
Имя Подпись	
Взам или №	
Подпись и дата	
Имя № подпись	

Изм.	Лист	№ докум.	Подпись	Дата

2.10. Руководство пользователя.

Банк Система
Внутренний портал сотрудников

Авторизация
Введите свои учётные данные

Логин: admin

Пароль:

Войти

Тестовые учётные записи

- Логин: admin / Пароль: admin
Администратор
- Логин: kasic22 / Пароль: kasic22
Сотрудник

Данные проверяются через MySQL

Безопасный вход • Все данные защищены

Рисунок 2.10 – «Окно авторизации с данными для входа администратора».

Главная Транзакции Клиенты **Админ-панель** Выйти

Добавить нового пользователя

ФИО: Иванов Иван Иванович Логин: ivan.ivanov Пароль: Роль: Выберите роль...

Добавить пользователя

Список пользователей Нажмите "*" для редактирования

ID	ФИО	Логин	Пароль	Роль	Действия
000001	admin	admin		admin	[Редактировать] [Удалить]
000002	Каминский Антон Александрович	kasic22		Сотрудник	[Редактировать] [Удалить]
000003	Пономарев Степан Иванович	keifad		Сотрудник	[Редактировать] [Удалить]

Рисунок 2.11 – «Окно админ-панели».

Главная Транзакции Клиенты **Админ-панель** **Выйти**

+ Добавить нового пользователя

ФИО **Логин** **Пароль** **Роль**

+ Добавить пользователя

Рисунок 2.12 – «Окно админ-панели создание нового пользователя».

Список пользователей Нажмите "✎" для редактирования

ID	ФИО	Логин	Пароль	Роль	Действия
000001	admin	admin	*****	admin	
000002	Каминский Антон Александрович	kasic22	*****	Сотрудник	
000003	Пономарев Степан Иванович	keifad	*****	Сотрудник	
000004	Иванов Иван Иванович	ivan.ivanov	*****	Сотрудник	

Рисунок 2.13 – «Окно админ-панели удаление пользователя».

Список пользователей Нажмите "✎" для редактирования

127.0.0.1:8000

Вы уверены, что хотите удалить этого пользователя?

Cancel OK

ID	ФИО	Логин	Пароль	Роль	Действия
000001	admin	admin	*****	admin	
000002	Каминский Антон Александрович	kasic22	*****	Сотрудник	
000003	Пономарев Степан Иванович	keifad	*****	Сотрудник	
000004	Иванов Иван Иванович	ivan.ivanov	*****	Сотрудник	

Рисунок 2.14 – «Окно админ-панели подтверждение удаления пользователя».

Главная Транзакции Клиенты **Админ-панель** **Выйти**

Здравствуйте, admin! admin 00:37:48

Последние транзакции 15 Нажмите для деталей

ID	Сумма	Отправитель	Получатель	Тип	Статус	Дата
#000078	5754.04 Р	Расчетный счет	Дебетовый счет	Перевод	Заблокирована	25.06.2026 00:36:50
#000077	9626.41 Р	Расчетный счет	Расчетный счет	Перевод	Выполнена	25.06.2026 00:36:50
#000076	9807.32 Р	Расчетный счет	Расчетный счет	Перевод	Выполнена	25.06.2026 00:36:50
#000075	9113.56 Р	Расчетный счет	Дебетовый счет	Оплата	Выполнена	25.06.2026 00:36:03
#000074	175.93 Р	Дебетовый счет	Расчетный счет	Перевод	Выполнена	25.06.2026 00:36:03
#000073	9652.11 Р	Расчетный счет	Дебетовый счет	Перевод	Выполнена	25.06.2026 00:36:03
#000072	262.61 Р	Дебетовый счет	Дебетовый счет	Оплата	Завершена	24.06.2026 21:08:07
#000071	424.95 Р	Дебетовый счет	Расчетный счет	Оплата	Выполнена	24.06.2026 21:08:07
#000070	6001.30 Р	Расчетный счет	Дебетовый счет	Перевод	Выполнена	24.06.2026 21:08:07

Статистика: Оплаты vs Переводы Всего

*Сравнение сумм оплат и переводов (тыс. Р)

Рисунок 2.15 – «Окно главной страницы администратора».

Подпись и дата
Имя Подпис
Взам или №
Подпись и дата
Имя № подл

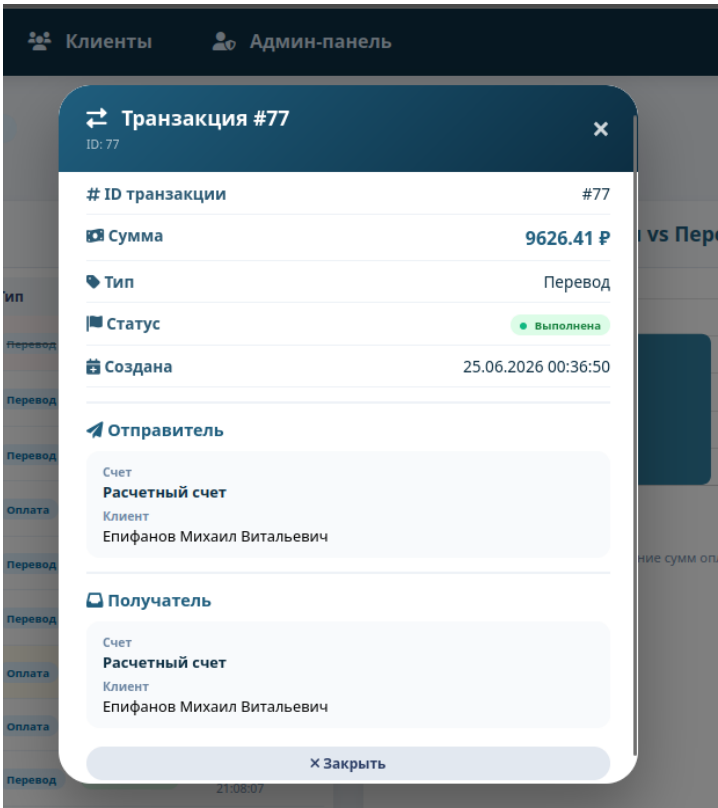


Рисунок 2.16 – «Окно деталей транзакции администратора».

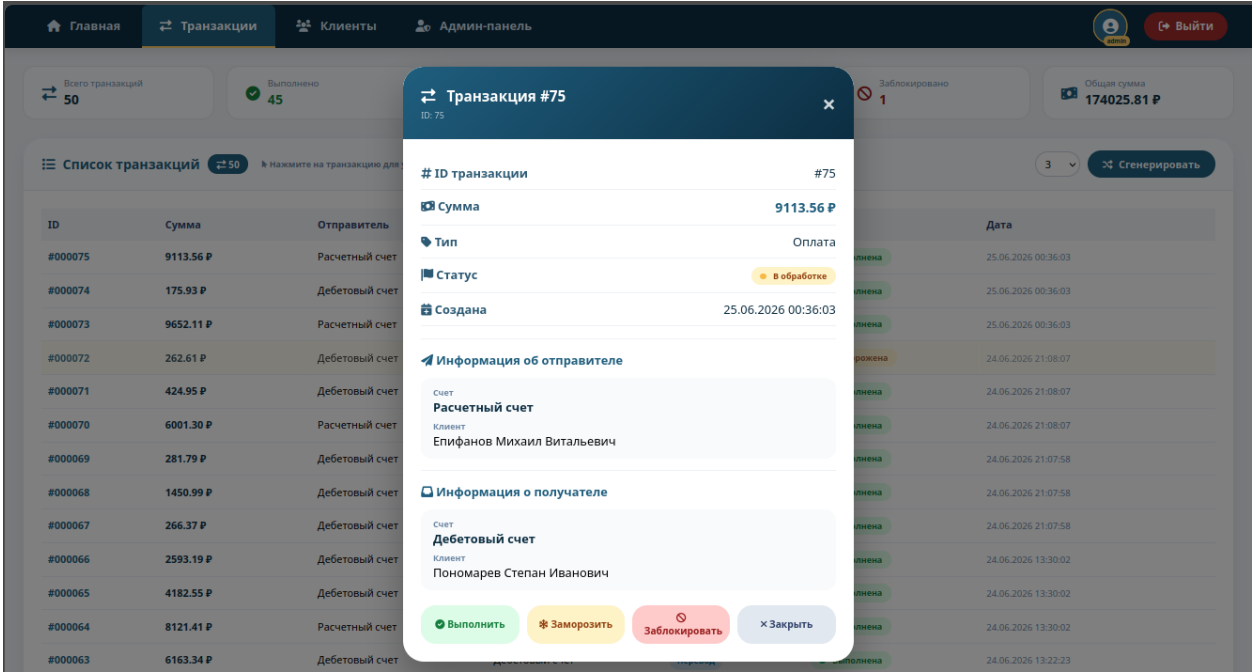


Рисунок 2.17 – «Окно управления транзакцией администратором».

Подпись и дата	
Имя Подписи	
Взам или №	
Подпись и дата	
Имя № подл	

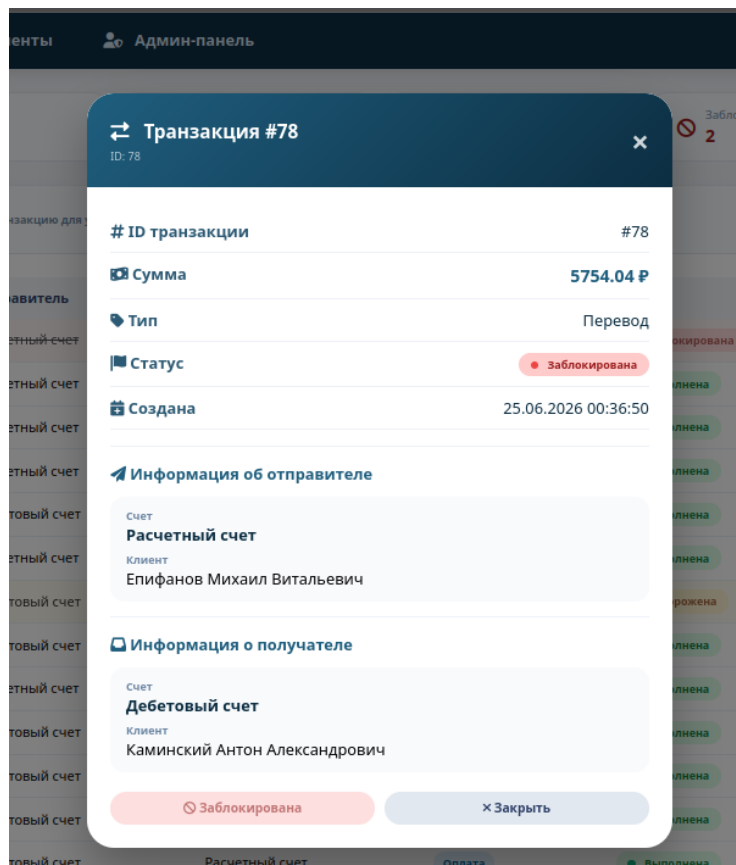


Рисунок 2.18 – «Окно блокировки транзакции администратором».

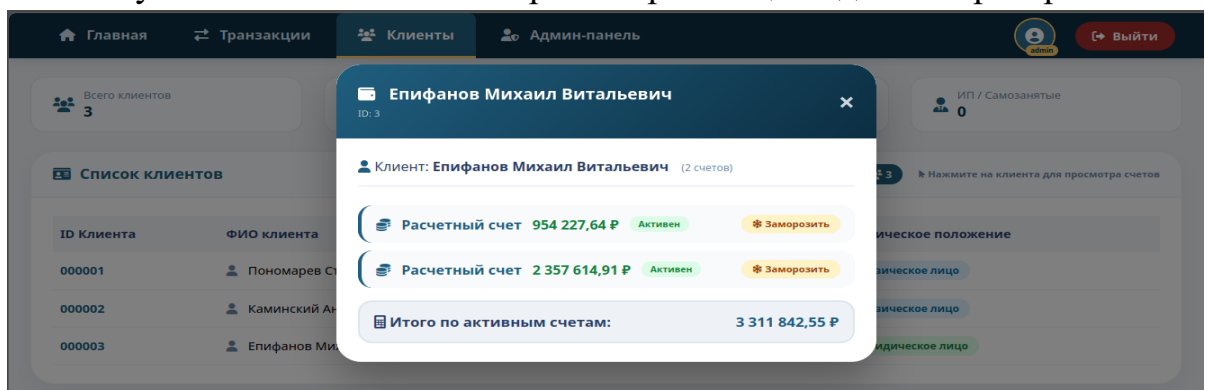


Рисунок 2.19 – «Окно счетов клиента администратора».

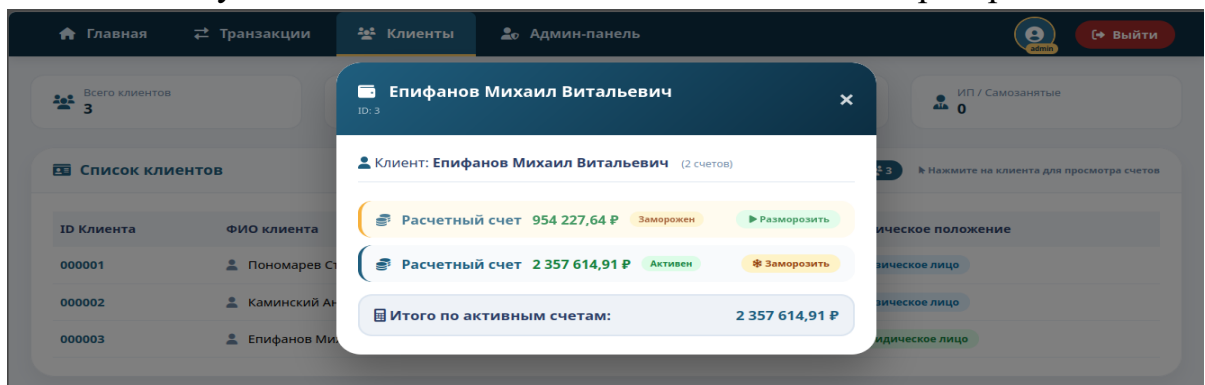


Рисунок 2.20 – «Окно заморозки счета клиента администратором».

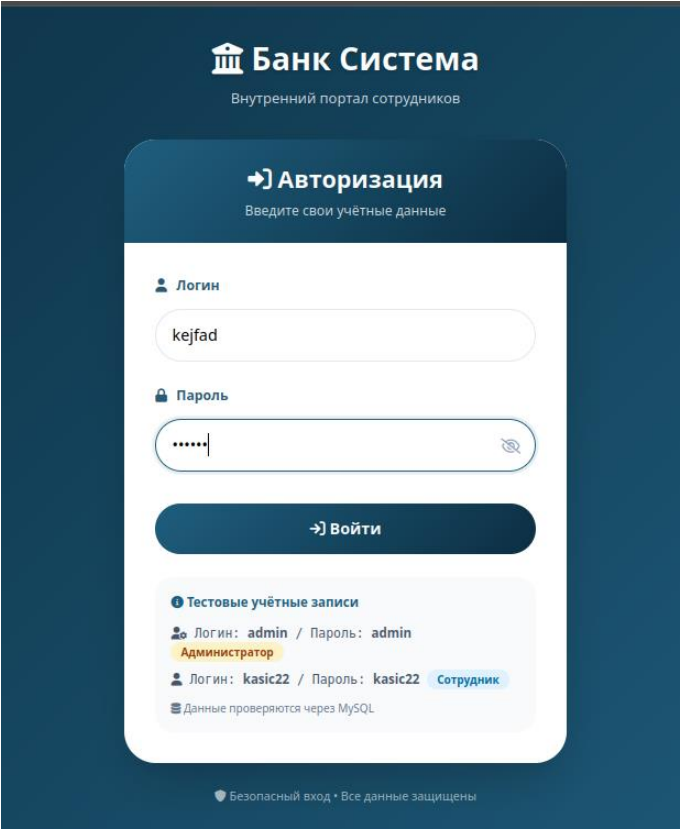


Рисунок 2.21 – «Окно авторизации с данными для входа сотрудника».

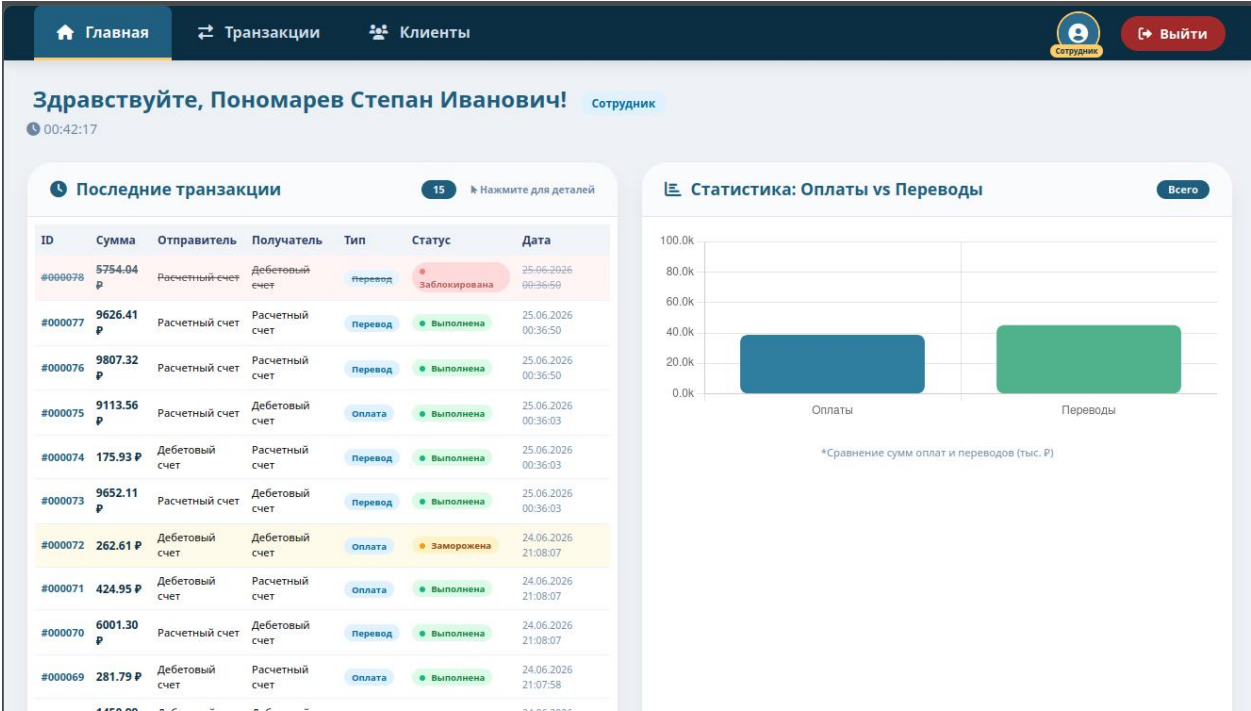


Рисунок 2.22 – «Главное окно сотрудника с дашбордом и диаграммой».

Подпись и дата	
Имя Начальн	
Взам илии Но	
Подпись и дата	
Имя № подл	

Изм.	Лист	№ докум.	Подпись	Дата

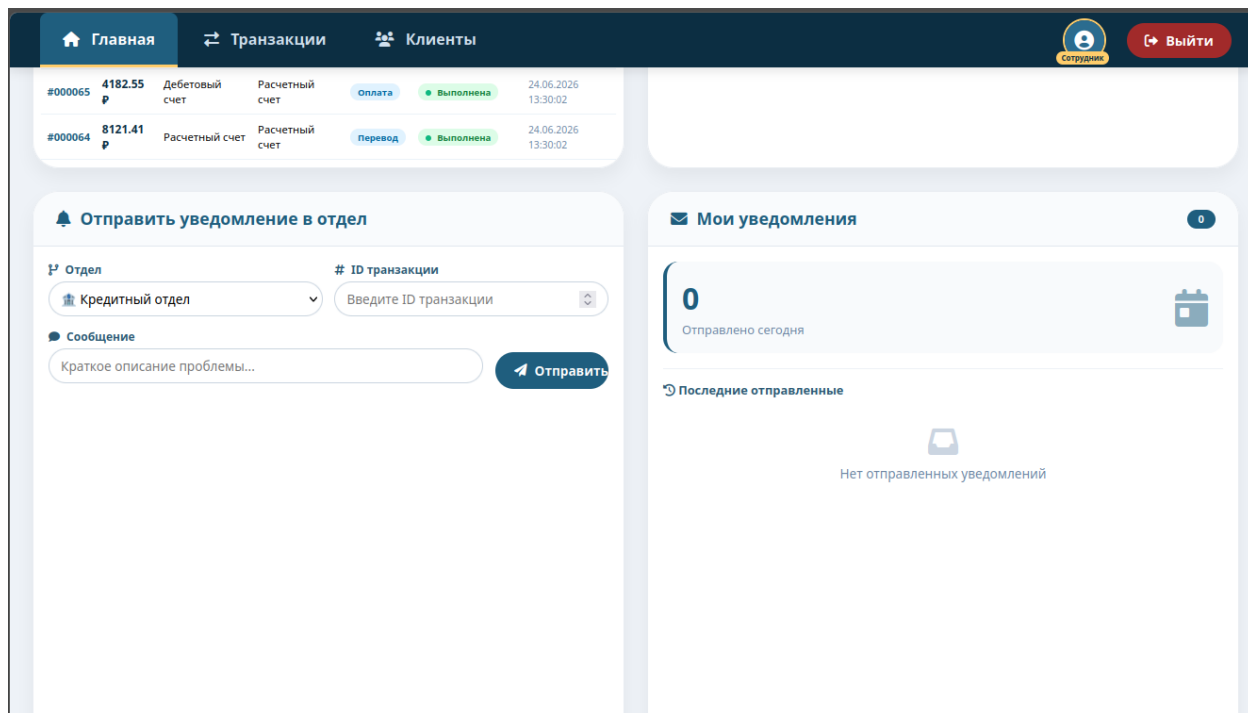


Рисунок 2.23 – «Главное окно сотрудника с блоками уведомлений».

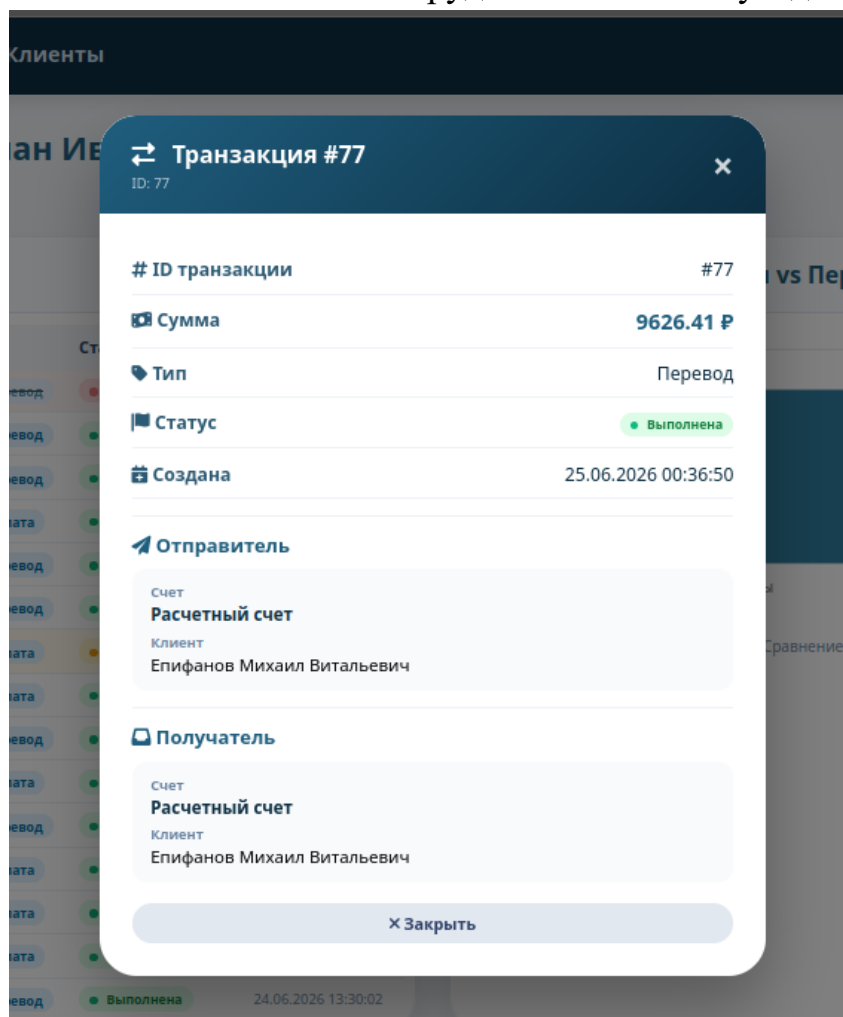


Рисунок 2.24 – «Окно деталей транзакции сотрудника».

Подпись и дата	
Имя Подпись	
Взам или №	
Подпись и дата	
Имя № подл	

Изм.	Лист	№ докум.	Подпись	Дата

Отправить уведомление в отдел

Отдел

Отдел безопасности

ID транзакции

78

Сообщение

Подозрительная операция

Отправить

Мои уведомления

1

Отправлено сегодня

Последние отправленные

Транзакция #77 — Кредитный отдел

00:44

Рисунок 2.25 – «Использование блока уведомлений сотрудником на главной странице».

Отправить уведомление в отдел

Отдел

Отдел безопасности

ID транзакции

Введите ID транзакции

Сообщение

Краткое описание проблемы...

Отправить

Уведомление отправлено в "Отдел безопасности" по транзакции #78

Мои уведомления

2

Отправлено сегодня

Последние отправленные

Транзакция #78 — Отдел безопасности

00:45

Транзакция #77 — Кредитный отдел

00:44

Рисунок 2.26 – «Появление уведомления в отчете сотрудника за день».

Главная

Транзакции

Клиенты

Выход

Всего клиентов

3

Физические лица

2

Юридические лица

1

ИП / Самозанятые

0

Список клиентов

3

Нажмите на клиента для просмотра счетов

ID Клиента	ФИО клиента	Юридическое положение
000001	Пономарев Степан Иванович	Физическое лицо
000002	Каминский Антон Александрович	Физическое лицо
000003	Епифанов Михаил Витальевич	Юридическое лицо

Рисунок 2.27 – «Окно клиентов сотрудника».

Подпись и дата	
Имя Подпись	
Взам или №	
Подпись и дата	
Имя №	

					ПП.09.02.07.491.19.26 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		49

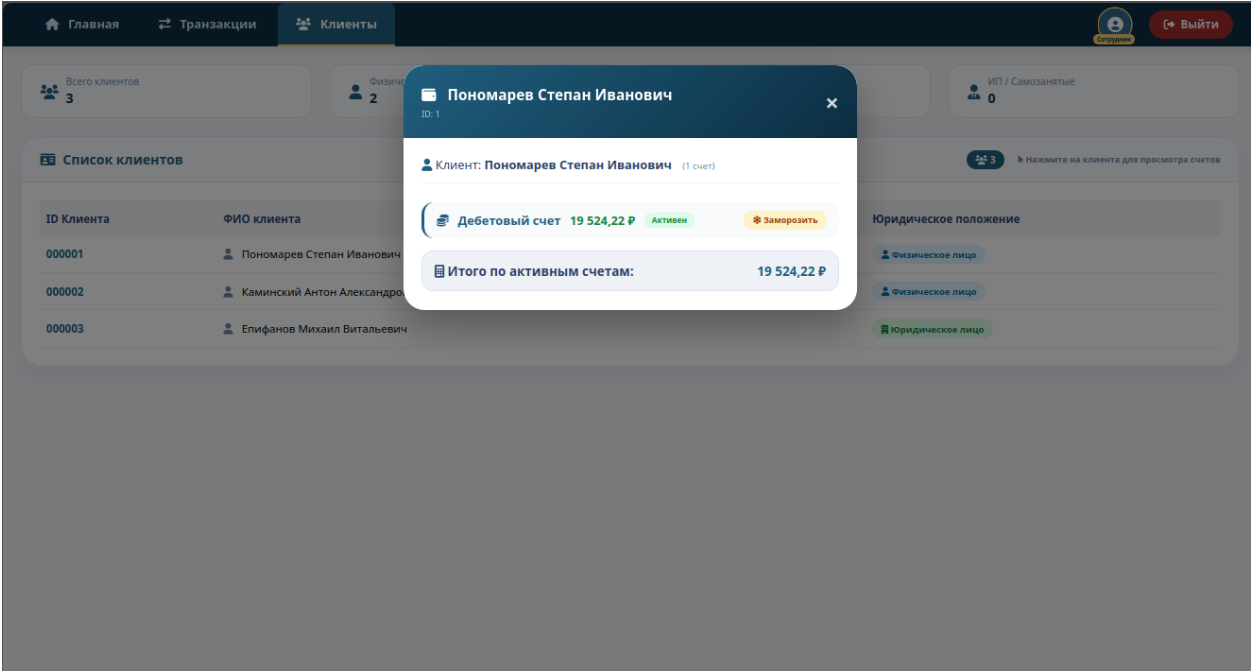


Рисунок 2.28 – «Окно просмотра счетов клиента сотрудником».

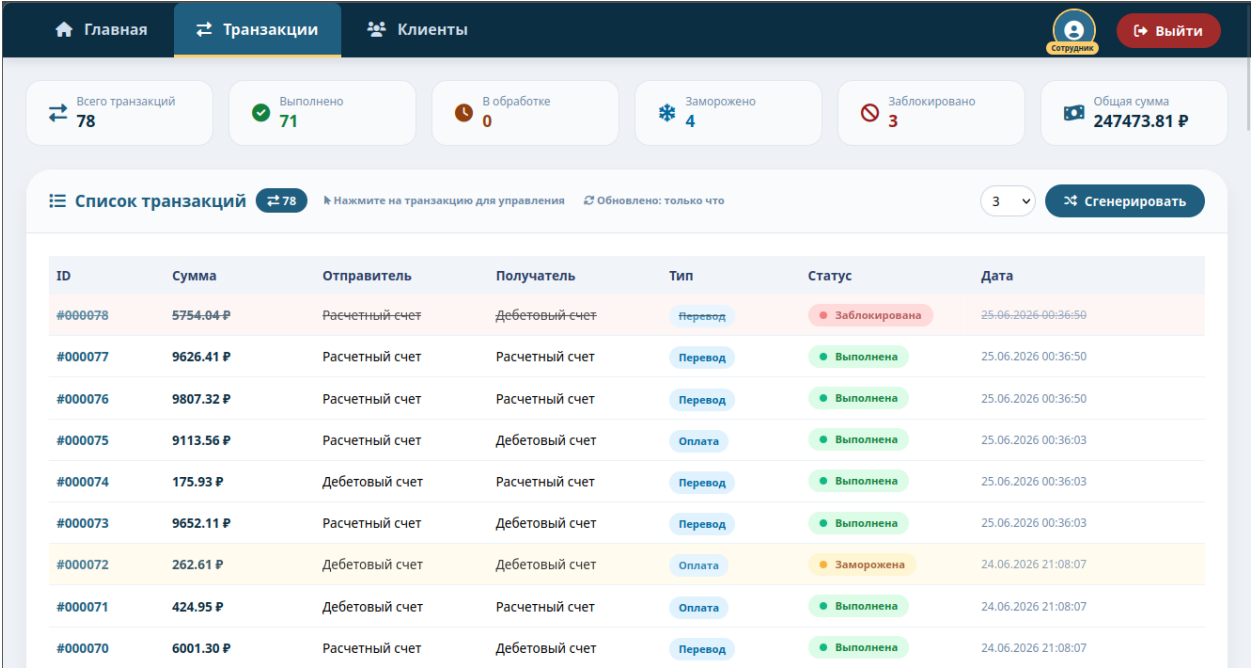


Рисунок 2.29 – «Окно транзакций сотрудника».

Подпись и дата	
Имя Начальника	
Взам. и дата	
Подпись и дата	
Имя № подл	

3. ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ ЭФФЕКТИВНОСТИ ПРОГРАММЫ

Данная задача относится к классу "Продуктов Web-дизайнов"

Накладные расходы (в %) - 25

Отчисления на социальные нужды (в %) - 30

Для расчета стоимости программного продукта используются данные о временных и стоимостных затратах на создание программного продукта из таблиц 3.1 – 3.2

Таблица 3.1 – Временные затраты на создание программного продукта

Этапы выполняемых работ	Время работы программиста, час	Время работы постановщика, час
Формирование требований ПП	10	2
Разработка концепции ПП	15	2
Постановка задачи	8	2
Структурирование данных, программы	80	2
Отладка	90	2
Оформление отчета	15	2
Ввод в действие	15	2
Сопровождение	7	2

Таблица 3.2 – Данные для расчета стоимости разработки программы

	Программист	Постановщик задачи
Среднемесячная зарплата, руб.	70000	65000
Количество раб. Дней в месяце	22	22
Продолжительность рабочего дня, час	8	8
Стоимость часа машинного времени, руб./час	461,87	431,47

Подпись и дата

Имя, Подпись

Взам или №

Подпись и дата

Имя, №, Подпись

ПП.09.02.07.491.19.26 ПЗ

Лист

52

Изм. Лист № докум. Подпись Дата

3.1 Основные расчетные формулы

3.1.1 Расчет средней стоимости одного часа работы

$$ЗЧ = (СРМЗП) / (КРДМ * ПРД), \text{руб.}, \quad (1)$$

где

ЗЧ – средняя стоимость часа работы, руб.;

СРМЗП – среднемесячная зарплата, руб.;

КРДМ – количество рабочих дней в месяце, дней;

ПРД – продолжительность рабочего дня, час.

Данные из таблицы 3.2.

3.1.2 Расчет расходов на оплату труда по программе

$$ЗП = ЗЧ * Т_{\text{прог}}, \text{руб.}, \quad (2)$$

где

ЗП – расходы на оплату труда по программе, руб.;

ЗЧ – средняя стоимость часа работника, руб.;

Т_{прог} – время подготовки программы, час.

Данные из таблицы 3.1 и таблицы 3.2.

3.1.3 Расчет отчислений на социальные нужды

$$СН = ЗП * (ПСН / 100), \text{руб.}, \quad (3)$$

где

СН – отчисления на социальные нужды, руб.;

ЗП – расходы на оплату труда по программе, руб.;

ПСН – процент отчислений на социальные нужды, %.

3.1.4 Расчет накладных расходов

$$НР = ЗП * (ПНР / 100), \text{руб.}, \quad (4)$$

Где

НР – накладные расходы, руб.;

ЗП – расходы на оплату труда по программе, руб.;

ПНР – процент накладных расходов, %.

3.1.5 Расчет оплаты машинного времени

$$МВ = Т_{\text{прм}} * СМЧ, \text{руб.}, \quad (5)$$

Где

МВ – оплата машинного времени, руб.;

Т_{прм} – время отладки программы, час.;

СМЧ – стоимость часа машинного времени, руб/час.

Данные из таблицы 3.1 и таблицы 3.2.

Подпись и дата	
Имя Подпись	
Взам или №	
Подпись и дата	
Имя № подл	

					ПП.09.02.07.491.19.26 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		53

3.1.6 Расчет стоимости программного продукта

$$СП = ЗП + СН + НР + МВ, \text{руб.}, \quad (6)$$

Где

СП – стоимость программного продукта, руб.;

ЗП – расходы на оплату труда, руб.;

СН – отчисления на социальные нужды, руб.;

НР – накладные расход, руб.;

МВ – стоимость часа машинного времени, руб.

Расчеты приведены в таблице 3.3.

3.2 Расчет стоимости программного продукта

Таблица 3.3 – Расчет стоимости программного продукта

	Программист	Постановщик задачи
Средняя стоимость 1 часа работы, руб.	397,73	369,32
Расходы на оплату труда по программе, руб.	95455,2	5909,12
Отчисление на социальные нужды, руб.	28636,56	1772,74
Накладные расходы, руб.	23863,8	1477,28
Оплата машинного времени, руб.	41568,3	862,94
Стоимость программного продукта, руб.	189523,86	10022,08

Суммарная стоимость программного продукта - 199545,94 руб.

Подпись и дата	
Имя	№ докум.
Взам	или №
Подпись и дата	
Имя	№ докум.

					ПП.09.02.07.491.19.26 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		54

3.3 Определение эффективности программного продукта

3.3.1 Определение годовых эксплуатационных затрат

$$\text{ЭЗ} = 31 + 32 + 33 + 34, \text{ руб.}, \quad (7)$$

где

ЭЗ – общие эксплуатационные затраты, руб.;

31 – годовой фонд заработной платы персонала, руб.;

32 – годовые амортизационные отчисления, руб.;

33 – годовые затраты на электроэнергию, руб.;

34 – прочие затраты (картриджи, диски, бумага и т.п.), руб.;

Расчеты приведены в таблице 3.4.

Таблица 3.4 – Расчет времени на обработку информации ручным и автоматизированным способами

Вид работы	Ручная обработка (день)	Ручная обработка (месяц)	Машинная обработка (день)	Машинная обработка (месяц)
Количество операций	7	154	7	154
Время на обработку одного документа	0,6		0,30	
Время на обработку всех документов	4,2	92,4	2,1	46,2
Итого часов		92,4		46,2
Итого часов в год		1108,8		554,4

Таблица 3.5 – Расчет годовых эксплуатационных затрат

Средняя месячная з\п работника, руб.	70000
Стоимость ЭВМ с ПО	100000
Коэффициент амортизации, в %	19
Мощность ЭВМ, кВтч	0,2
Стоимость кВтч	6,4
Прочие расходы	1800
Коэффициент К	0,054

Подпись и дата

Имя Подпись

Взам или №

Подпись и дата

Имя № подл

3.3.2 Вычисление годовых эксплуатационных затрат

- 1) Годовой фонд заработной платы персонала, который занимается обслуживанием ЭВМ:

$$31 = q * C_3 / 26 / t * (k+1), \text{ руб.}, \quad (8)$$

где

q – время работы с системой в месяц, час;

t – продолжительность рабочего дня, час;

C₃ – средняя месячная заработная плата работника

$$31 = 12 * 46,2 * 70000 / 26 / 8 * (0,054 + 1) = 16387,67 \text{ руб.}$$

- 2) Годовые амортизационные отчисления:

$$32 = C_{ЭВМ} * K, \text{ руб.}, \quad (9)$$

где

C_{ЭВМ} – стоимость ЭВМ с установленным программным обеспечением, руб.

K – нормативный коэффициент амортизации - 19%

$$32 = 100000 * 19 / 100 = 19000 \text{ руб.}$$

- 3) Годовые затраты на электроэнергию:

$$33 = KЧ_{ЭВМ} * М_{ЭВМ} * Ц_{КВТ}, \text{ руб.}, \quad (10)$$

где

33 – затраты на электроэнергию за год, руб.;

KЧ_{ЭВМ} – количество часов работы ЭВМ в год, руб.;

М_{ЭВМ} – мощность ЭВМ, кВтч.;

Ц_{КВТ} – стоимость одного кВтч, руб.

$$33 = 554,4 * 0,2 * 6,4 = 709,63 \text{ руб.}$$

- 4) Прочие затраты:

$$34 = C_{ЭВМ} * 0,1, \text{ руб.}, \quad (11)$$

где

34 – прочие затраты, руб.;

C_{ЭВМ} – Стоимость ЭВМ с установленным программным обеспечением, руб.

$$34 = 1800 * 0,1 \text{ руб.}$$

- 5) Годовые эксплуатационные затраты:

$$\Sigma 3 = 16387,67 + 19000 + 709,63 + 1800 = 46097,3 \text{ руб.}$$

Имя	Подпись и дата
Имя	Подпись
Имя	Подпись
Имя	Подпись
Имя	Подпись

Изм.	Лист	№ докум.	Подпись	Дата	ПП.09.02.07.491.19.26 ПЗ	Лист
						56

3.3.3 Расчет ожидаемого годового экономического эффекта

$$\mathcal{E} = \mathcal{E}_1 + \mathcal{E}_2 - \mathcal{E}_3, \text{ руб.}, \quad (12)$$

где

$\mathcal{E}$ – общая экономия;

$\mathcal{E}_1$ – экономия от снижения стоимости обработки информации;

$\mathcal{E}_2$ – экономия от увеличения производительности труда;

$\mathcal{E}_3$ – годовые эксплуатационные затраты, руб.

$$\mathcal{E}_1 = (PO - AO) * Cp, \text{ руб.}, \quad (13)$$

где

АО – время на автоматическую обработку информации за год = 554,4 час;

РО – время на ручную обработку информации за год = 1108,8 час;

Cp – среднечасовая ставка работника = 397,73 руб./час.

$$\mathcal{E}_1 = (1108,8 - 554,4) * 397,73 = 220501,512 \text{ руб.}$$

$$\mathcal{E}_2 = OCPO - OCAO, \text{ руб.}, \quad (14)$$

где

$\mathcal{E}_2$ – экономия от увеличения производительности труда, руб.;

OCPO – общая стоимость ожидания запросов за год при ручной обработке,

руб.;

OCAO – общая стоимость ожидания запросов за год при автоматизированной,

руб.

$$\mathcal{E}_2 = 20045,592 - 10873,044 = 9172,548 \text{ руб.}$$

Расчеты приведены в таблице 3.6.

Таблица 3.6 – Расчет ожидаемого экономического эффекта

	Ручная обработка	Автоматизированная обработка
Среднее количество запросов в месяц	7	7
Среднечасовая з\п работника, руб.	397,73	431,47
Среднее время ожидания на запрос, час.	0,6	0,30
Среднее время ожидания в месяц, час.	4,2	2,1
Стоимость времени ожидания в месяц, руб.	1670,466	906,087
Стоимость времени ожидания в год, руб.	20045,592	10873,044
Экономия рабочего времени в год, руб.		9172,548

Экономия от увеличения производительности труда составляет 9172,548 руб.

Общая экономия с учетом эксплуатационных расходов:

$$\mathcal{E} = 220501,512 + 9172,548 - 46097,3 = 183576,76 \text{ руб.}$$

3.3.4. Срок окупаемости капиталовложений

$$T = L / \mathcal{E}, \text{ год}, \quad (15)$$

где

T – срок окупаемости капиталовложений, год;

L – дополнительные капиталовложения (стоимость программного продукта);

$\mathcal{E}$ – общая экономия от использования программного продукта.

$$T = 199545,94 / 183576,76 = 1,09 \text{ год.}$$

Подпись и дата	
Имя Подпись	
Взам или №	
Подпись и дата	
Имя № подл	

					ПП.09.02.07.491.19.26 ПЗ	Лист
						57
Изм.	Лист	№ докум.	Подпись	Дата		

4. ОХРАНА ТРУДА.

1. Цель и задачи раздела

Цель — проанализировать требования охраны труда, предъявляемые к рабочим местам специалистов в области информационных систем и программирования, и сформулировать практические рекомендации по обеспечению безопасных и комфортных условий труда.

Задачи:

Рассмотреть нормативно-правовую базу в области охраны труда.

Охарактеризовать вредные и опасные производственные факторы, воздействующие на разработчика программного обеспечения.

Описать требования к организации рабочего места, эргономике, режиму труда и отдыха.

Сформулировать мероприятия по профилактике профессионального выгорания и сохранению здоровья сотрудников.

2. Нормативно-правовое обеспечение охраны труда

Деятельность специалиста по информационным системам и программированию регулируется следующими основными документами:

Трудовой кодекс Российской Федерации (раздел X «Охрана труда», ст. 209–231);

ГОСТ 12.0.003-2015 «ССБТ. Опасные и вредные производственные факторы. Классификация»;

СанПиН 1.2.3685-21 «Гигиенические нормативы и требования к обеспечению безопасности и (или) безвредности для человека факторов среды обитания»;

СП 52.13330.2016 «Естественное и искусственное освещение» (актуализированная редакция СНиП 23-05-95);

СанПиН 2.2.2/2.4.1340-03 «Гигиенические требования к персональным электронно-вычислительным машинам и организации работы»;

ГОСТ 12.1.005-88 «Общие санитарно-гигиенические требования к воздуху рабочей зоны»;

Правила по охране труда при работе с инструментом и приспособлениями, утверждённые приказом Минтруда России;

Федеральный закон № 426-ФЗ «О специальной оценке условий труда».

Соблюдение перечисленных документов позволяет работодателю обеспечить правовую защищённость работника, а работнику — сохранить здоровье в процессе профессиональной деятельности.

3. Характеристика условий труда программиста

Труд разработчика программного обеспечения относится к категории работ, связанных с длительной умственной нагрузкой, высокой концентрацией внимания и продолжительным пребыванием за персональным компьютером.

Подпись и дата						
Имя	Подпись					
Взам	Имя	№				
Подпись и дата						
Имя	№					
Изм.	Лист	№ докум.	Подпись	Дата	ПП.09.02.07.491.19.26 ПЗ	Лист
						58

Согласно Руководству Р 2.2.2006-05 и СанПиН 1.2.3685-21, выделяют следующие вредные и опасные производственные факторы, характерные для данной профессии:

Группа факторов

Конкретные проявления

Физические

Недостаточное или избыточное освещение, электромагнитное излучение, статическое электричество, шум систем охлаждения, неблагоприятный микроклимат

Химические

Выделение вредных веществ из пластика, лакокрасочных покрытий, расходных материалов (при обслуживании оргтехники)

Психофизиологические

Монотонность, информационные перегрузки, умственное перенапряжение, стресс, нарушение режима труда и отдыха

Эргономические

Неправильная организация рабочего места, неудобная мебель, вынужденная рабочая поза

4. Требования к организации рабочего места

4.1. Расположение и компоновка

Рабочее место программиста должно обеспечивать удобное выполнение всех предусмотренных функций без лишних перемещений. Согласно СанПиН 2.2.2/2.4.1340-03, необходимо соблюдать следующие минимальные расстояния:

расстояние от экрана монитора до глаз пользователя — не менее 600–700 мм; расстояние между боковыми поверхностями мониторов — не менее 1 200 мм;

площадь рабочего места — не менее 4,5 м² (для мониторов на базе плоских дискретных экранов — ЖК-мониторов);

высота рабочей поверхности стола — 680–800 мм (регулируемая).

4.2. Мебель.

Кресло оператора должно быть подъёмно-поворотным, регулируемым по высоте и углам наклона спинки, с подлокотниками. Конструкция кресла должна поддерживать рациональную рабочую позу, позволять изменять позу для снижения статического напряжения мышц.

4.3. Освещение

Освещённость на поверхности стола в зоне размещения рабочего документа должна составлять 300–500 лк. Рекомендуется использовать комбинированное освещение (общее + местное). Применение только местного освещения не допускается, так как создаёт резкий перепад

Подпись и дата	
Имя	Подпись
Взам или №	
Подпись и дата	
Имя	Подпись

					ПП.09.02.07.491.19.26 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		

не загромождать пути эвакуации и подходы к средствам пожаротушения; исключить применение открытого огня и нагревательных приборов с открытой спиралью.

5.3. Электромагнитная безопасность

Уровни электромагнитного излучения от мониторов и системных блоков нормируются в соответствии с СанПиН 2.2.4.3359-16. Временные допустимые уровни электромагнитных полей устанавливаются по результатам производственного контроля и специальной оценки условий труда (СОУТ). При превышении нормативов рекомендуется использовать мониторы с пониженным уровнем излучения и обеспечивать расстояние не менее 1,2 м от боковых и задних стенок соседних мониторов.

6. Режим труда и отдыха

Длительная непрерывная работа за компьютером приводит к развитию компьютерного зрительного синдрома, мышечно-скелетных нарушений и психоэмоционального утомления. Для их профилактики СанПиН 2.2.2/2.4.1340-03 устанавливает следующий режим: при непрерывной работе за компьютером не более 1 часа необходимо делать перерыв;

суммарное время работы за экраном — не более 6 часов за смену;

рекомендуемая продолжительность перерывов — 10–15 минут каждые 45–60 минут работы;

во время перерывов целесообразно выполнять гимнастику для глаз, комплексы упражнений для шеи и плечевого пояса.

Кроме того, работникам, профессионально связанным с разработкой программного обеспечения, рекомендуется проходить ежегодные профилактические медицинские осмотры, включая осмотр офтальмолога, невролога и терапевта.

7. Психосоциальные аспекты и профилактика выгорания

Гуманизация охраны труда предполагает внимание не только к физическим, но и к психосоциальным факторам. Специфика работы программиста — дедлайны, многозадачность, неопределённость требований, длительная изоляция в цифровой среде — создаёт предпосылки для развития профессионального выгорания (синдром эмоционального истощения, деперсонализации и редукции личных достижений, описанный К. Маслач).

В целях сохранения психологического здоровья рекомендуется:

формировать сбалансированный график работы с чередованием задач различной сложности;

Инд. №	№ докум.	Подпись и дата	Взам. инв. №	Имя	Подпись и дата	Лист
Изм.	Лист	№ докум.	Подпись	Дата	ПП.09.02.07.491.19.26 ПЗ	61

создавать в коллективе атмосферу психологической безопасности и открытой коммуникации;

предоставлять работникам возможность участвовать в принятии решений, касающихся организации их труда;

внедрять программы well-being (корпоративного благополучия): доступ к спортивным занятиям, комнаты отдыха, гибкий график;

проводить тренинги по управлению стрессом и тайм-менеджменту.

8. Мероприятия по улучшению условий труда

На основании проведённого анализа можно предложить следующий комплекс мероприятий:

Организационные — разработка и внедрение стандарта предприятия по охране труда для ИТ-специалистов; проведение регулярных инструктажей и обучения; формирование культуры безопасного поведения.

Технические — модернизация парка компьютерной техники с использованием мониторов, сертифицированных по уровню электромагнитного излучения; установка источников бесперебойного питания для защиты оборудования и данных.

Эргономические — обеспечение рабочих мест регулируемой мебелью, ортопедическими креслами, подставками для ног; организация правильного освещения с использованием светодиодных светильников с регулируемой цветовой температурой.

Медико-профилактические — организация ежегодных медосмотров, обеспечение доступа к корпоративному психологу, проведение «зарядок для глаз» в начале рабочего дня.

Социальные — поддержание баланса между работой и личной жизнью, развитие системы гибких льгот, поощрение инициатив сотрудников по улучшению условий труда.

9. Выводы

Охрана труда специалиста в области информационных систем и программирования — это комплексная задача, выходящая за рамки традиционного понимания «техники безопасности». Она охватывает вопросы эргономики, санитарно-гигиенического благополучия, психологического здоровья и организационной культуры. Гуманизация подхода к охране труда предполагает переход от формального соблюдения нормативов к созданию такой рабочей среды, в которой сотрудник чувствует себя защищённым, уважаемым и мотивированным к профессиональному развитию.

Применение предложенных мероприятий позволит:

снизить уровень профессиональных заболеваний и травматизма;

повысить производительность и качество выполняемой работы;

Подпись и дата	
Имя и фамилия	
Взам или №	
Подпись и дата	
Имя и фамилия	

Имя	№ докум.	Подпись	Дата	Лист
Изм.	Лист	№ докум.	Подпись	Дата
ПП.09.02.07.491.19.26 ПЗ				62

Таким образом, инвестиции в охрану труда в ИТ-отрасли — это не только выполнение юридических обязательств, но и стратегический вклад в устойчивое развитие компании и благополучие её сотрудников.

Үнх № тогтол Подпись и дата Рзам или № Илч № Подпись и дата						ПП.09.02.07.491.19.26 ПЗ	Лист
							63
	Изм.	Лист	№ докум.	Подпись	Дата		

ЗАКЛЮЧЕНИЕ

В ходе прохождения практики по теме "Мониторинг банковских операций" мне удалось получить ценный опыт в области разработки и внедрения систем автоматического контроля за финансовыми транзакциями. Практическая деятельность позволила лучше понять особенности работы банковских сервисов, а также важность обеспечения безопасности и своевременного выявления подозрительных действий.

Особое внимание уделялось развитию навыков анализа данных и использованию современных технологий для повышения эффективности мониторинга. В результате выполнения поставленных задач я смог не только укрепить свои технические знания, но и понять, насколько важна команда профессионалов, работающих над защитой финансовых систем.

Данное практическое наставление подтвердило мое стремление развиваться в области информационных технологий и бизнес-аналитики в банковском секторе. Надеюсь, полученные знания станут надежным фундаментом для дальнейших профессиональных достижений и внесут вклад в развитие финансовой безопасности.

Подпись и дата	
Имя Подписи	
Взам или №	
Подпись и дата	
Имя № подл	

					ПП.09.02.07.491.19.26 ПЗ	Лист
						64
Изм.	Лист	№ докум.	Подпись	Дата		

Приложение А. Листинг приложения

```
from django.shortcuts import render, redirect
from django.contrib import messages
from django.contrib.auth import login, logout
from django.contrib.auth.models import User
from django.http import JsonResponse
from django.db import connection
from django.db.utils import OperationalError
from django.contrib.auth.hashers import make_password, check_password
import hashlib
from .models import Users_db, Clients_db;
import json
import random
import threading
import time
from datetime import datetime, timedelta
```

```
# Create your views here.
```

```
def index(request):
    return render(request, "index.html")
```

```
def transactions(request):
    return render(request, "pages/transactions.html")
```

```
def clients(request):
    return render(request, "pages/clients.html")
```

```
def admin_panel(request):
    return render(request, "pages/admin-panel.html")
```

```
def log_in(request):
    return render(request, "pages/authorization.html")
```

```
def clients(request):
    # Проверка авторизации
    if not request.session.get('is_authenticated'):
        messages.warning(request, '⚠️ Пожалуйста, авторизуйтесь')
    return redirect('authorization')
```

Подпись и дата

Имя Начальн

Взам илии Но

Подпись и дата

Инд № подл

ПП.09.02.07.491.19.26 ПЗ

Лист

65

Изм. Лист № докум. Подпись Дата

```
clients_list = []
```

```
error = None
```

```
# Счетчики для статистики
```

```
total_count = 0
```

```
individual_count = 0 # Физические лица
```

```
legal_count = 0      # Юридические лица
```

```
ip_count = 0         # ИП
```

```
self_count = 0       # Самозанятые
```

```
unknown_count = 0    # Не указано
```

```
try:
```

```
    # Получаем всех клиентов с уровнями
```

```
    clients_queryset
```

```
Clients_db.Clients.objects.select_related('Levels_of_clients_id_level').all()
```

```
    for client in clients_queryset:
```

```
        level_name = client.Levels_of_clients_id_level.name_level if  
client.Levels_of_clients_id_level else 'Не указано'
```

```
        clients_list.append({  
            'id': client.id_clients,  
            'name': client.name_clients,  
            'level': level_name,  
            'level_id': client.Levels_of_clients_id_level_id  
        })
```

```
# Подсчет статистики
```

```
total_count += 1
```

```
if level_name == 'Физическое лицо':
```

```
    individual_count += 1
```

```
elif level_name == 'Юридическое лицо':
```

```
    legal_count += 1
```

```
elif level_name == 'ИП':
```

```
    ip_count += 1
```

```
elif level_name == 'Самозанятый':
```

```
    self_count += 1
```

```
else:
```

```
    unknown_count += 1
```

Подпись и дата

Имя Подписи

Взам или №

Подпись и дата

Имя № подл

ПП.09.02.07.491.19.26 ПЗ

Лист

66

Изм. Лист № докум. Подпись Дата

```
print(f"👤 Найдено клиентов: {len(clients_list)}")
```

```
except Exception as e:
```

```
    error = str(e)
```

```
    print(f"❌ Ошибка загрузки клиентов: {error}")
```

```
    messages.error(request, f"❌ Ошибка загрузки данных: {error}")
```

```
context = {
```

```
    'clients': clients_list,
```

```
    'user_name': request.session.get('user_name'),
```

```
    'user_role': request.session.get('user_role'),
```

```
    'clients_count': total_count,
```

```
    'individual_count': individual_count,
```

```
    'legal_count': legal_count,
```

```
    'ip_count': ip_count,
```

```
    'self_count': self_count,
```

```
    'unknown_count': unknown_count,
```

```
    'error': error,
```

```
}
```

```
return render(request, 'pages/clients.html', context)
```

```
def get_client_wallets(request):
```

```
    """API для получения счетов клиента по его ID"""
```

```
    if request.method != 'GET':
```

```
        return JsonResponse({'success': False, 'error': 'Invalid request method'})
```

```
    if not request.session.get('is_authenticated'):
```

```
        return JsonResponse({'success': False, 'error': 'Not authorized'})
```

```
    client_id = request.GET.get('client_id')
```

```
    if not client_id:
```

```
        return JsonResponse({'success': False, 'error': 'ID клиента не указан!'})
```

```
    try:
```

```
        from django.db import connections
```

```
        with connections['second'].cursor() as cursor:
```

```
            # Получаем информацию о клиенте
```

ПП.09.02.07.491.19.26 ПЗ

Лист

67

Подпись и дата

Имя Подписи

Взам или №

Подпись и дата

Имя № подл

Изм.

Лист

№ докум.

Подпись

Дата

```

cursor.execute("""
    SELECT id_clients, name_clients
    FROM Clients
    WHERE id_clients = %s
""", [client_id])
client = cursor.fetchone()

if not client:
    return JsonResponse({'success': False, 'error': 'Клиент не найден!'})

client_id_db, client_name = client

# Получаем все счета клиента
cursor.execute("""
    SELECT id_wallets, name_wallets, amount_wallets, status
    FROM Wallets
    WHERE Clients_id_clients = %s
""", [client_id_db])

wallets = cursor.fetchall()

wallets_data = []
total_amount = 0

status_display_map = {
    'active': 'Активен',
    'frozen': 'Заморожен',
    'blocked': 'Заблокирован'
}

for wallet in wallets:
    wallet_id, wallet_name, wallet_amount, wallet_status = wallet
    wallet_status = wallet_status if wallet_status else 'active'

    wallets_data.append({
        'id': wallet_id,
        'name': wallet_name,
        'amount': float(wallet_amount),
        'status': wallet_status,

```

Получить и дата

Имя Номер

Взам или №

Получить и дата

Имя №

```

        'status_display': status_display_map.get(wallet_status, wallet_status),
        'is_active': wallet_status == 'active'
    })
    if wallet_status == 'active':
        total_amount += float(wallet_amount)

    return JsonResponse({
        'success': True,
        'client_name': client_name,
        'wallets': wallets_data,
        'total_amount': total_amount,
        'wallets_count': len(wallets_data)
    })

except Exception as e:
    print(f" ❌ Ошибка получения счетов: {e}")
    return JsonResponse({'success': False, 'error': str(e)})

```

```

def toggle_wallet_status(request):
    """API для заморозки/разморозки счета"""
    if request.method != 'POST':
        return JsonResponse({'success': False, 'error': 'Invalid request method'})

    if not request.session.get('is_authenticated'):
        return JsonResponse({'success': False, 'error': 'Not authorized'})

    # Проверка прав доступа (только администратор)
    if request.session.get('user_role') != 'admin':
        return JsonResponse({'success': False, 'error': 'Access denied. Только для администраторов!'})

    try:
        data = json.loads(request.body)
        wallet_id = data.get('wallet_id')
        action = data.get('action')

        if not wallet_id or not action:

```

Підписи і дата

Ім'я Начальн

Рзам і ім'я №

Підписи і дата

Ім'я № розділ

действие!'}))

```
wallet = cursor.fetchone()
```

```
return JsonResponse({'success': False, 'error': 'Счет не найден!'})
```

Определяем новый статус

```
if current_status == 'frozen':
```

```
if current_status == 'blocked':
```

```
return JsonResponse({'success': False, 'error': 'Нельзя заморозить
```

```
message = f'Счет "{wallet_name}" успешно заморожен!'
```

```
if current_status != 'frozen':
```

```
new_status = 'active'
```

else:

Обновляем статус

```
cursor.execute("UPDATE Wallets SET status = %s WHERE id = %s")
```

'frozen': 'Заморожен',

```

        'blocked': 'Заблокирован'
    }

```

```

return JsonResponse({
    'success': True,
    'message': message,
    'new_status': new_status,
    'new_status_display': status_display_map.get(new_status, new_status)
})

```

except Exception as e:

```

    print(f" ❌ Ошибка изменения статуса счета: {e}")
    return JsonResponse({'success': False, 'error': str(e)})

```

def get_transactions_update(request):

```

    """API для получения обновленных транзакций (для polling)"""

```

```

    if request.method != 'GET':

```

```

        return JsonResponse({'success': False, 'error': 'Invalid request method'})

```

```

    if not request.session.get('is_authenticated'):

```

```

        return JsonResponse({'success': False, 'error': 'Not authorized'})

```

```

    try:

```

```

        from django.db import connections

```

```

        with connections['second'].cursor() as cursor:

```

```

            cursor.execute("""

```

```

                SELECT

```

```

                    t.id_transactions,

```

```

                    t.amount_transaction,

```

```

                    t.sender_id,

```

```

                    t.recipient_id,

```

```

                    t.Types_of_transactions_id_type_transactions,

```

```

                    t.status,

```

```

                    t.created_at,

```

```

                    s.name_wallets as sender_wallet,

```

```

                    r.name_wallets as recipient_wallet,

```

```

                    cl_s.name_clients as sender_client,

```

```

                    cl_r.name_clients as recipient_client

```

```

                FROM Transactions t

```

Подпись и дата

Имя Подпись

Взам или №

Подпись и дата

Имя № подл

```

LEFT JOIN Wallets s ON t.sender_id = s.id_wallets
LEFT JOIN Wallets r ON t.recipient_id = r.id_wallets
LEFT JOIN Clients cl_s ON s.Clients_id_clients = cl_s.id_clients
LEFT JOIN Clients cl_r ON r.Clients_id_clients = cl_r.id_clients
ORDER BY t.id_transactions DESC
LIMIT 50
""")
transactions_raw = cursor.fetchall()

```

```

status_display_map = {
    'pending': 'В обработке',
    'completed': 'Выполнена',
    'frozen': 'Заморожена',
    'blocked': 'Заблокирована'
}

```

```

# Получаем типы транзакций
cursor.execute("SELECT id_type_transactions, name_type_transactions
FROM Types_of_transactions")
types_raw = cursor.fetchall()
type_map = {}
for type_row in types_raw:
    type_map[type_row[0]] = type_row[1]

```

```

transactions_list = []
for tx in transactions_raw:
    status = tx[5] if tx[5] else 'pending'
    type_id = tx[4] if tx[4] else None
    type_name = type_map.get(type_id, 'Не указан') if type_id else 'Не
указан'

```

```

transactions_list.append({
    'id': tx[0],
    'amount': float(tx[1]) if tx[1] else 0,
    'sender_id': tx[2],
    'recipient_id': tx[3],
    'type': type_name,
    'status': status,
    'status_display': status_display_map.get(status, status),

```

Подпись и дата	
Имя Подписи	
Взам или №	
Подпись и дата	
Имя № подписи	

					ПП.09.02.07.491.19.26 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		72


```

        'created_at': tx[6].strftime('%d.%m.%Y %H:%M:%S') if tx[6] else '—',
        'sender_wallet': tx[7] if tx[7] else '—',
        'recipient_wallet': tx[8] if tx[8] else '—',
        'sender_client': tx[9] if tx[9] else '—',
        'recipient_client': tx[10] if tx[10] else '—'
    })

```

```

    return JsonResponse({
        'success': True,
        'transactions': transactions_list
    })

```

except Exception as e:

```

    print(f" ❌ Ошибка получения обновлений транзакций: {e}")
    return JsonResponse({'success': False, 'error': str(e)})

```

```

def process_transaction_async(transaction_id, sender_id, recipient_id, amount):
    """Асинхронно обрабатывает транзакцию с задержкой 5 секунд"""
    from django.db import connections

```

```

    try:

```

```

        # Ждем 5 секунд
        time.sleep(5)

```

```

        with connections['second'].cursor() as cursor:

```

```

            # Проверяем, что транзакция все еще в статусе 'pending'

```

```

            cursor.execute("SELECT status FROM Transactions WHERE
id_transactions = %s", [transaction_id])

```

```

            result = cursor.fetchone()

```

```

            if result and result[0] == 'pending':

```

```

                # Обновляем балансы

```

```

                cursor.execute("""

```

```

                    UPDATE Wallets

```

```

                    SET amount_wallets = amount_wallets - %s

```

```

                    WHERE id_wallets = %s

```

```

                """, [amount, sender_id])

```

```

            cursor.execute("""

```

Имя	Подпись и дата
Имя	Подпись
Взам	Имя
Имя	Подпись и дата
Имя	Подпись

Изм.	Лист	№ докум.	Подпись	Дата	ПП.09.02.07.491.19.26 ПЗ	Лист
						73

```

UPDATE Wallets
SET amount_wallets = amount_wallets + %s
WHERE id_wallets = %s
""" , [amount, recipient_id])

```

```

# Обновляем статус на 'completed'
cursor.execute("""
    UPDATE Transactions
    SET status = 'completed'
    WHERE id_transactions = %s
    """, [transaction_id])

```

```

print(f"✅ Транзакция #{transaction_id} выполнена через 5 секунд")
else:

```

```

print(f"⚠️ Транзакция #{transaction_id} уже не в статусе 'pending'")

```

```

except Exception as e:

```

```

print(f"❌ Ошибка обработки транзакции #{transaction_id}: {e}")

```

```

def generate_random_transaction():

```

```

    """Генерирует случайную транзакцию (только Оплата или Перевод)"""
    from django.db import connections

```

```

    try:

```

```

        with connections['second'].cursor() as cursor:
            # Получаем все активные счета с балансом > 200
            cursor.execute("""

```

```

                SELECT id_wallets, amount_wallets, Clients_id_clients
                FROM Wallets
                WHERE status = 'active' AND amount_wallets > 200
            """)

```

```

            sender_wallets = cursor.fetchall()

```

```

            if not sender_wallets:

```

```

                print(f"⚠️ Нет счетов с достаточным балансом для создания транзакций")

```

```

                return None

```

Подпись и дата

Имя Подписи

Взам или №

Подпись и дата

Имя № подл

```

# Выбираем случайного отправителя
sender = random.choice(sender_wallets)
sender_id, sender_balance, sender_client = sender
sender_balance = float(sender_balance)

# Получаем все активные счета (кроме счета отправителя)
cursor.execute("""
    SELECT id_wallets, amount_wallets, Clients_id_clients
    FROM Wallets
    WHERE status = 'active'
    AND id_wallets != %s
""", [sender_id])
recipient_wallets = cursor.fetchall()

if not recipient_wallets:
    print("⚠ Нет других счетов для получения транзакций")
    return None

# Выбираем случайного получателя
recipient = random.choice(recipient_wallets)
recipient_id, recipient_balance, recipient_client = recipient
recipient_balance = float(recipient_balance)

# Генерируем случайную сумму
max_amount = min(10000, sender_balance * 0.5)
min_amount = 100

if max_amount < min_amount:
    print(f"⚠ Баланс отправителя слишком мал: {sender_balance}")
    return None

amount = round(random.uniform(min_amount, max_amount), 2)

# Выбираем случайный тип транзакции (1 - Оплата, 2 - Перевод)
type_id = random.randint(1, 2)

# Получаем названия счетов для отображения

```

```

        cursor.execute("SELECT name_wallets FROM Wallets WHERE id_wallets
= %s", [sender_id])
        sender_name = cursor.fetchone()[0]

        cursor.execute("SELECT name_wallets FROM Wallets WHERE id_wallets
= %s", [recipient_id])
        recipient_name = cursor.fetchone()[0]

        # Получаем имена клиентов
        if sender_client:
            cursor.execute("SELECT name_clients FROM Clients WHERE
id_clients = %s", [sender_client])
            sender_client_name = cursor.fetchone()
            sender_client_name = sender_client_name[0] if sender_client_name else
'Неизвестный клиент'
        else:
            sender_client_name = 'Неизвестный клиент'

        if recipient_client:
            cursor.execute("SELECT name_clients FROM Clients WHERE
id_clients = %s", [recipient_client])
            recipient_client_name = cursor.fetchone()
            recipient_client_name = recipient_client_name[0] if
recipient_client_name else 'Неизвестный клиент'
        else:
            recipient_client_name = 'Неизвестный клиент'

        return {
            'amount': amount,
            'sender_id': sender_id,
            'recipient_id': recipient_id,
            'type_id': type_id,
            'sender_name': sender_name,
            'recipient_name': recipient_name,
            'sender_client': sender_client_name,
            'recipient_client': recipient_client_name,
            'sender_balance': sender_balance,
            'recipient_balance': recipient_balance
        }

```

Имя	Подпись и дата
Имя	Подпись
Взам или №	
Подпись и дата	
Имя	Подпись

					ПП.09.02.07.491.19.26 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		76

except Exception as e:

print(f" ❌ Ошибка генерации транзакции: {e}")

return None

def execute_transaction(transaction_data):

"""Создает транзакцию со статусом 'pending' и запускает асинхронную обработку"""

from django.db import connections

if not transaction_data:

return False

try:

with connections['second'].cursor() as cursor:

Проверяем баланс отправителя

cursor.execute("SELECT amount_wallets FROM Wallets WHERE
id_wallets = %s", [transaction_data['sender_id']])

current_balance = cursor.fetchone()

if not current_balance:

return False

current_balance = float(current_balance[0])

if current_balance < transaction_data['amount']:

print(f" ⚠ Недостаточно средств! Баланс: {current_balance}, Сумма:
{transaction_data['amount']}")

return False

Создаем транзакцию со статусом 'pending'

cursor.execute("""

INSERT INTO Transactions (

amount_transaction,

sender_id,

recipient_id,

Types_of_transactions_id_type_transactions,

status,

created_at

)

```
VALUES (%s, %s, %s, %s, 'pending', NOW())
''', [
    transaction_data['amount'],
    transaction_data['sender_id'],
    transaction_data['recipient_id'],
    transaction_data['type_id']
])
```

```
transaction_id = cursor.lastrowid
print(f"🔄 Создана транзакция #{transaction_id} со статусом 'pending'")
```

```
# Запускаем асинхронную обработку транзакции (через 5 секунд)
thread = threading.Thread(
    target=process_transaction_async,
    args=(transaction_id, transaction_data['sender_id'],
transaction_data['recipient_id'], transaction_data['amount'])
)
thread.daemon = True # Поток завершится при завершении основного
процесса
thread.start()

print(f"⌚ Запущен таймер 5 секунд для транзакции #{transaction_id}")
return True
```

```
except Exception as e:
    print(f"❌ Ошибка выполнения транзакции: {e}")
    return False
```

```
def generate_transactions(request):
    """API для генерации случайных транзакций"""
    if request.method != 'POST':
        return JsonResponse({'success': False, 'error': 'Invalid request method'})

    if not request.session.get('is_authenticated'):
        return JsonResponse({'success': False, 'error': 'Not authorized'})

    try:
        data = json.loads(request.body)
```

Подпись и дата	
Имя Подписи	
Взам или №	
Подпись и дата	
Имя № подл	

					ПП.09.02.07.491.19.26 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		78

```
count = data.get('count', 1)
```

```
if count < 1 or count > 10:
```

```
    return JsonResponse({'success': False, 'error': 'Количество транзакций  
должно быть от 1 до 10'})
```

```
created_count = 0
```

```
failed_count = 0
```

```
transactions_info = []
```

```
for i in range(count):
```

```
    transaction_data = generate_random_transaction()
```

```
    if transaction_data:
```

```
        if execute_transaction(transaction_data):
```

```
            created_count += 1
```

```
            transactions_info.append({
```

```
                'amount': transaction_data['amount'],
```

```
                'sender': transaction_data['sender_name'],
```

```
                'recipient': transaction_data['recipient_name']
```

```
            })
```

```
        else:
```

```
            failed_count += 1
```

```
    else:
```

```
        failed_count += 1
```

```
    message = f"✅ Создано {created_count} транзакций (будут выполнены  
через 5 секунд)"
```

```
    if failed_count > 0:
```

```
        message += f", {failed_count} не удалось создать"
```

```
    return JsonResponse({
```

```
        'success': True,
```

```
        'message': message,
```

```
        'created_count': created_count,
```

```
        'failed_count': failed_count,
```

```
        'transactions': transactions_info
```

```
    })
```

Подпись и дата

Имя Начальн

Взам илии №

Подпись и дата

Имя № докум

```
except Exception as e:
```

```
    print(f" ❌ Ошибка генерации транзакций: {e}")
```

```
    return JsonResponse({'success': False, 'error': str(e)})
```

```
def transactions(request):
```

```
    """Страница транзакций"""
```

```
    if not request.session.get('is_authenticated'):
```

```
        messages.warning(request, ' ⚠️ Пожалуйста, авторизуйтесь')
```

```
        return redirect('authorization')
```

```
    transactions_list = []
```

```
    error = None
```

```
    stats = {
```

```
        'total': 0,
```

```
        'pending': 0,
```

```
        'completed': 0,
```

```
        'frozen': 0,
```

```
        'blocked': 0,
```

```
        'total_amount': 0
```

```
    }
```

```
    try:
```

```
        from django.db import connections
```

```
        with connections['second'].cursor() as cursor:
```

```
            cursor.execute("""
```

```
                SELECT
```

```
                    t.id_transactions,
```

```
                    t.amount_transaction,
```

```
                    t.sender_id,
```

```
                    t.recipient_id,
```

```
                    t.Types_of_transactions_id_type_transactions,
```

```
                    t.status,
```

```
                    t.created_at,
```

```
                    s.name_wallets as sender_wallet,
```

```
                    r.name_wallets as recipient_wallet,
```

```
                    cl_s.name_clients as sender_client,
```

```
                    cl_r.name_clients as recipient_client
```

```
                FROM Transactions t
```

Підписи і дата

Ім'я Наблюд

Взам ім'я №

Підписи і дата

Ім'я № набл


```

LEFT JOIN Wallets s ON t.sender_id = s.id_wallets
LEFT JOIN Wallets r ON t.recipient_id = r.id_wallets
LEFT JOIN Clients cl_s ON s.Clients_id_clients = cl_s.id_clients
LEFT JOIN Clients cl_r ON r.Clients_id_clients = cl_r.id_clients
ORDER BY t.id_transactions DESC

```

```

"""

```

```

transactions_raw = cursor.fetchall()

```

```

status_display_map = {
    'pending': 'В обработке',
    'completed': 'Выполнена',
    'frozen': 'Заморожена',
    'blocked': 'Заблокирована'
}

```

```

# Получаем типы транзакций

```

```

cursor.execute("SELECT id_type_transactions, name_type_transactions
FROM Types_of_transactions")

```

```

types_raw = cursor.fetchall()

```

```

type_map = {}

```

```

for type_row in types_raw:

```

```

    type_map[type_row[0]] = type_row[1]

```

```

for tx in transactions_raw:

```

```

    status = tx[5] if tx[5] else 'pending'

```

```

    stats['total'] += 1

```

```

    stats[status] = stats.get(status, 0) + 1

```

```

    stats['total_amount'] += float(tx[1]) if tx[1] else 0

```

```

    type_id = tx[4] if tx[4] else None

```

```

    type_name = type_map.get(type_id, 'Не указан') if type_id else 'Не
указан'

```

```

transactions_list.append({

```

```

    'id': tx[0],

```

```

    'amount': float(tx[1]) if tx[1] else 0,

```

```

    'sender_id': tx[2],

```

```

    'recipient_id': tx[3],

```

```

    'type': type_name,

```

Идентификатор документа	Подпись и дата
Имя подписи	
Взам или №	
Подпись и дата	
Идентификатор документа	

Изм.	Лист	№ докум.	Подпись	Дата	ПП.09.02.07.491.19.26 ПЗ	Лист
						81

```

        'status': status,
        'status_display': status_display_map.get(status, status),
        'created_at': tx[6].strftime('%d.%m.%Y %H:%M:%S') if tx[6] else '—',
        'sender_wallet': tx[7] if tx[7] else '—',
        'recipient_wallet': tx[8] if tx[8] else '—',
        'sender_client': tx[9] if tx[9] else '—',
        'recipient_client': tx[10] if tx[10] else '—'
    })

```

```

print(f"

```

```

except Exception as e:
    error = str(e)
    print(f"❌ Ошибка загрузки транзакций: {error}")
    messages.error(request, f"❌ Ошибка загрузки данных: {error}")

```

```

context = {
    'transactions': transactions_list,
    'stats': stats,
    'user_name': request.session.get('user_name'),
    'user_role': request.session.get('user_role'),
    'transactions_count': len(transactions_list),
    'error': error,
}
return render(request, 'pages/transactions.html', context)

```

```

def get_transaction_details(request):
    """API для получения деталей транзакции"""
    if request.method != 'GET':
        return JsonResponse({'success': False, 'error': 'Invalid request method'})

    if not request.session.get('is_authenticated'):
        return JsonResponse({'success': False, 'error': 'Not authorized'})

    transaction_id = request.GET.get('transaction_id')

```

Подпись и дата	
Имя Подписи	
Результат	
Подпись и дата	
Имя Подписи	

					ПП.09.02.07.491.19.26 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		82


```

else:
    type_name = 'Не указан'

status_display_map = {
    'pending': 'В обработке',
    'completed': 'Выполнена',
    'frozen': 'Заморожена',
    'blocked': 'Заблокирована'
}

return JsonResponse({
    'success': True,
    'id': tx[0],
    'amount': float(tx[1]) if tx[1] else 0,
    'sender_id': tx[2],
    'recipient_id': tx[3],
    'type': type_name,
    'status': tx[5] if tx[5] else 'pending',
    'status_display': status_display_map.get(tx[5], tx[5]),
    'created_at': tx[6].strftime('%d.%m.%Y %H:%M:%S') if tx[6] else '—',
    'sender_wallet': tx[7] if tx[7] else '—',
    'recipient_wallet': tx[8] if tx[8] else '—',
    'sender_client': tx[9] if tx[9] else '—',
    'recipient_client': tx[10] if tx[10] else '—'
})

```

```

except Exception as e:
    print(f" ❌ Ошибка получения деталей транзакции: {e}")
    return JsonResponse({'success': False, 'error': str(e)})

```

```

def toggle_transaction_status(request):
    """API для заморозки/разморозки/блокировки транзакции"""
    if request.method != 'POST':
        return JsonResponse({'success': False, 'error': 'Invalid request method'})

    if not request.session.get('is_authenticated'):
        return JsonResponse({'success': False, 'error': 'Not authorized'})

```

Идентификатор документа	Подпись и дата
Имя пользователя	Имя пользователя
Взам или №	Взам или №
Подпись и дата	Подпись и дата
Имя пользователя	Имя пользователя

Изм.	Лист	№ докум.	Подпись	Дата	ПП.09.02.07.491.19.26 ПЗ	Лист
						84

```

if request.session.get('user_role') != 'admin':
    return JsonResponse({'success': False, 'error': 'Access denied. Только для администраторов!'})

```

```

try:
    data = json.loads(request.body)
    transaction_id = data.get('transaction_id')
    action = data.get('action')

```

```

if not transaction_id or not action:
    return JsonResponse({'success': False, 'error': 'Не указан ID транзакции или действие!'})

```

```

from django.db import connections
with connections['second'].cursor() as cursor:
    cursor.execute("SELECT id_transactions, status FROM Transactions WHERE id_transactions = %s", [transaction_id])
    tx = cursor.fetchone()

```

```

if not tx:
    return JsonResponse({'success': False, 'error': 'Транзакция не найдена!'})

```

```

current_status = tx[1]

```

```

if action == 'freeze':
    if current_status == 'frozen':
        return JsonResponse({'success': False, 'error': 'Транзакция уже заморожена!'})

```

```

    if current_status == 'blocked':
        return JsonResponse({'success': False, 'error': 'Нельзя заморозить заблокированную транзакцию!'})

```

```

    if current_status == 'completed':
        return JsonResponse({'success': False, 'error': 'Нельзя заморозить выполненную транзакцию!'})

```

```

    new_status = 'frozen'

```

```

    message = 'Транзакция успешно заморожена!'

```

```

elif action == 'unfreeze':

```

```

    if current_status != 'frozen':

```

Підписи і дата

Ім'я Надібн

Взам ім'я №

Підписи і дата

Ім'я № наділ

```

        return JsonResponse({'success': False, 'error': 'Транзакция не
заморожена!'})
        new_status = 'pending'
        message = 'Транзакция успешно разморожена!'
    elif action == 'block':
        if current_status == 'blocked':
            return JsonResponse({'success': False, 'error': 'Транзакция уже
заблокирована!'})
        if current_status == 'completed':
            return JsonResponse({'success': False, 'error': 'Нельзя заблокировать
выполненную транзакцию!'})
        new_status = 'blocked'
        message = 'Транзакция успешно заблокирована!'
    elif action == 'complete':
        if current_status == 'completed':
            return JsonResponse({'success': False, 'error': 'Транзакция уже
выполнена!'})
        if current_status in ['frozen', 'blocked']:
            return JsonResponse({'success': False, 'error': 'Нельзя выполнить
замороженную или заблокированную транзакцию!'})
        new_status = 'completed'
        message = 'Транзакция успешно выполнена!'
    else:
        return JsonResponse({'success': False, 'error': 'Неизвестное действие!'})

```

```

        cursor.execute("UPDATE Transactions SET status = %s WHERE
id_transactions = %s", [new_status, transaction_id])

```

```

status_display_map = {
    'pending': 'В обработке',
    'completed': 'Выполнена',
    'frozen': 'Заморожена',
    'blocked': 'Заблокирована'
}

```

```

return JsonResponse({
    'success': True,
    'message': message,
    'new_status': new_status,

```

Имя	Подпись и дата
Имя	Подпись
Взам или №	
Имя	Подпись и дата
Имя №	подп

					ПП.09.02.07.491.19.26 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		86

```
'new_status_display': status_display_map.get(new_status, new_status)
}))
```

except Exception as e:

```
print(f" ✖ Ошибка изменения статуса транзакции: {e}")
return JsonResponse({'success': False, 'error': str(e)})
```

```
def authorization(request):
```

```
    if request.session.get('is_authenticated'):
        return redirect('main')
```

```
    if request.method == 'POST':
```

```
        login_input = request.POST.get('login')
        password_input = request.POST.get('password')
```

```
    if not login_input or not password_input:
```

```
        messages.error(request, ' ✖ Введите логин и пароль!')
        return render(request, 'pages/authorization.html')
```

```
    try:
```

```
        with connection.cursor() as cursor:
```

```
            cursor.execute("""
```

```
                SELECT u.id_users, u.name_user, u.login_user, u.password_user,
r.name_role
```

```
                FROM Users u
```

```
                LEFT JOIN Roles r ON u.Roles_id_roles = r.id_roles
```

```
                WHERE u.login_user = %s
```

```
            """, [login_input])
```

```
            user = cursor.fetchone()
```

```
    if user:
```

```
        user_id, name_user, login_user, password_hash, role_name = user
```

```
        # Проверяем пароль с помощью check_password
```

```
        if check_password(password_input, password_hash):
```

```
            request.session['user_id'] = user_id
```

```
            request.session['user_name'] = name_user
```

```
            request.session['user_login'] = login_user
```

Подпись и дата

Имя Подпись

Взам или №

Подпись и дата

Имя № подл

ПП.09.02.07.491.19.26 ПЗ

Лист

87

Изм. Лист № докум. Подпись Дата

```

        request.session['user_role'] = role_name if role_name else
        'Сотрудник'
        request.session['is_authenticated'] = True

        messages.success(request, f'✅ Добро пожаловать, {name_user}!')
        return redirect('main')
    else:
        messages.error(request, '❌ Неверный пароль!')
    else:
        messages.error(request, '❌ Пользователь с таким логином не
        найден!')

    except Exception as e:
        messages.error(request, f'❌ Ошибка: {str(e)}')
        print(f'Ошибка авторизации: {e}')

    return render(request, 'pages/authorization.html')

return render(request, 'pages/authorization.html')

```

```

def add_user(request):
    if request.method != 'POST':
        return JsonResponse({'success': False, 'error': 'Invalid request method'})

    if not request.session.get('is_authenticated'):
        return JsonResponse({'success': False, 'error': 'Not authorized'})

    if request.session.get('user_role') != 'admin':
        return JsonResponse({'success': False, 'error': 'Access denied'})

    try:
        data = json.loads(request.body)
        full_name = data.get('fullName', "").strip()
        login = data.get('login', "").strip()
        password = data.get('password', "").strip()
        role_id = data.get('roleId')
    
```

Подпись и дата

Имя Подпись

Взам или №

Подпись и дата

Имя № подл


```

if request.session.get('user_role') != 'admin':
    return JsonResponse({'success': False, 'error': 'Access denied'})

```

```

try:
    data = json.loads(request.body)
    user_id = data.get('userId')
    full_name = data.get('fullName', "").strip()
    login = data.get('login', "").strip()
    password = data.get('password', "").strip()
    role_id = data.get('roleId')

```

```

    if not user_id or not full_name or not login or not role_id:
        return JsonResponse({'success': False, 'error': 'Все поля обязательны для
заполнения!'})

```

```

    if password and len(password) < 3:
        return JsonResponse({'success': False, 'error': 'Пароль должен быть не
менее 3 символов!'})

```

```

    with connection.cursor() as cursor:
        cursor.execute("SELECT COUNT(*) FROM Users WHERE login_user =
%s AND id_users != %s", [login, user_id])
        count = cursor.fetchone()[0]
        if count > 0:
            return JsonResponse({'success': False, 'error': 'Пользователь с таким
логинем уже существует!'})

```

```

    # Если пароль передан, хэшируем его
    if password:
        hashed_password = make_password(password)
        cursor.execute("""
            UPDATE Users
            SET name_user = %s, login_user = %s, password_user = %s,
Roles_id_roles = %s
            WHERE id_users = %s
            """, [full_name, login, hashed_password, role_id, user_id])
    else:
        cursor.execute("""

```

Подпись и дата	
Имя Подписи	
Взам или №	
Подпись и дата	
Имя № подл	

					ПП.09.02.07.491.19.26 ПЗ	Лист
						90
Изм.	Лист	№ докум.	Подпись	Дата		

```

UPDATE Users
SET name_user = %s, login_user = %s, Roles_id_roles = %s
WHERE id_users = %s
""", [full_name, login, role_id, user_id])

```

```

return JsonResponse({'success': True, 'message': 'Пользователь успешно
обновлен!'})

```

```

except Exception as e:

```

```

    print(f" ❌ Ошибка обновления: {e}")

```

```

    return JsonResponse({'success': False, 'error': str(e)})

```

```

def main(request):

```

```

    if not request.session.get('is_authenticated'):

```

```

        messages.warning(request, ' ⚠️ Пожалуйста, авторизуйтесь')

```

```

        return redirect('authorization')

```

```

    context = {

```

```

        'user_name': request.session.get('user_name'),

```

```

        'user_role': request.session.get('user_role'),

```

```

    }

```

```

    return render(request, 'index.html', context)

```

```

def admin_panel(request):

```

```

    # Проверка авторизации

```

```

    if not request.session.get('is_authenticated'):

```

```

        messages.warning(request, ' ⚠️ Пожалуйста, авторизуйтесь')

```

```

        return redirect('authorization')

```

```

    # Проверка роли - ТОЛЬКО АДМИНИСТРАТОР

```

```

    if request.session.get('user_role') != 'admin':

```

```

        messages.error(request, ' 🚫 Доступ запрещен! Только для
администраторов')

```

```

        return redirect('main')

```

```

    users = []

```

Поліпівський

Ім'я

Вік

Підпис

№ докум.

```
roles = []
```

```
try:
```

```
    users_queryset = Users_db.Users.objects.select_related('Roles_id_roles').all()
```

```
    for user in users_queryset:
```

```
        role_name = user.Roles_id_roles.name_role if user.Roles_id_roles else  
'Сотрудник'
```

```
        users.append({
```

```
            'id': user.id_users,
```

```
            'name': user.name_user,
```

```
            'login': user.login_user,
```

```
            'password': user.password_user,
```

```
            'role': role_name,
```

```
            'role_id': user.Roles_id_roles_id
```

```
        })
```

```
roles_queryset = Users_db.Roles.objects.all()
```

```
for role in roles_queryset:
```

```
    roles.append({
```

```
        'id': role.id_roles,
```

```
        'name': role.name_role
```

```
    })
```

```
except Exception as e:
```

```
    messages.error(request, f'❌ Ошибка загрузки данных: {str(e)}')
```

```
context = {
```

```
    'users': users,
```

```
    'roles': roles,
```

```
    'user_name': request.session.get('user_name'),
```

```
    'user_role': request.session.get('user_role'),
```

```
}
```

```
return render(request, 'pages/admin-panel.html', context)
```

```
def delete_user(request):
```

```
    if request.method != 'POST':
```

```
        return JsonResponse({'success': False, 'error': 'Invalid request method'})
```

```
# Проверка авторизации
```

Подпись и дата	
Имя Подписи	
Взам или №	
Подпись и дата	
Имя № подл	

					ПП.09.02.07.491.19.26 ПЗ	Лист
						92
Изм.	Лист	№ докум.	Подпись	Дата		

```

if not request.session.get('is_authenticated'):
    return JsonResponse({'success': False, 'error': 'Not authorized'})

# Проверка роли
if request.session.get('user_role') != 'admin':
    return JsonResponse({'success': False, 'error': 'Access denied'})

try:
    data = json.loads(request.body)
    user_id = data.get('userId')

    if not user_id:
        return JsonResponse({'success': False, 'error': 'ID пользователя не
указан!'})

    # Не даем удалить самого себя
    if user_id == request.session.get('user_id'):
        return JsonResponse({'success': False, 'error': 'Нельзя удалить самого
себя!'})

    # Находим и удаляем пользователя
    user = Users_db.Users.objects.get(id_users=user_id)
    user.delete()

    return JsonResponse({'success': True, 'message': 'Пользователь успешно
удален!'})

except Exception as e:
    print(f" ❌ Ошибка удаления: {e}")
    return JsonResponse({'success': False, 'error': str(e)})

def logout_view(request):
    request.session.flush()
    messages.info(request, '👋 Вы вышли из системы')
    return redirect('authorization')

```

Имя	Подпись и дата
Имя	Подпись
Взам или №	
Подпись и дата	
Имя	Подпись

					ПП.09.02.07.491.19.26 ПЗ
Изм.	Лист	№ докум.	Подпись	Дата	

Приложение Б. Техническое задание

1. Назначение документа

Настоящее техническое задание определяет требования к разработке веб-платформы банковского сервиса, предназначенной для автоматизации мониторинга и анализа, и предотвращения мошеннических действий в финансовых операциях.

2. Наименование разработки

Система: BankMonitor — веб-платформа банковского сервиса

Краткое описание: многопользовательская веб-система, обеспечивающая мониторинг банковских операций.

3 Назначение

Веб-платформа BankMonitor предназначена для предоставления банковских операций клиентов банка сотрудникам банка.

4 Цели создания

Обеспечение безопасности и законности банковских операций.

Внедрение ролевой модели для разграничения полномочий пользователей

Повышение эффективности работы банковских сотрудников

5. Задачи системы

Регистрация и авторизация пользователей

Мониторинг банковских операций

Ролевое разграничение доступа

Формирование отчётности

6. Требования к безопасности

Хранение паролей в хешированном виде (bcrypt)

Защита от SQL-инъекций

Ограничение попыток авторизации (5 попыток, блокировка на 15 минут)

Требование сложности пароля (минимум 8 символов, буквы и цифры)

Подпись и дата	
Имя, Подпись	
Взам или №	
Подпись и дата	
Имя, №, Подпись	

					ПП.09.02.07.491.19.26 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		94